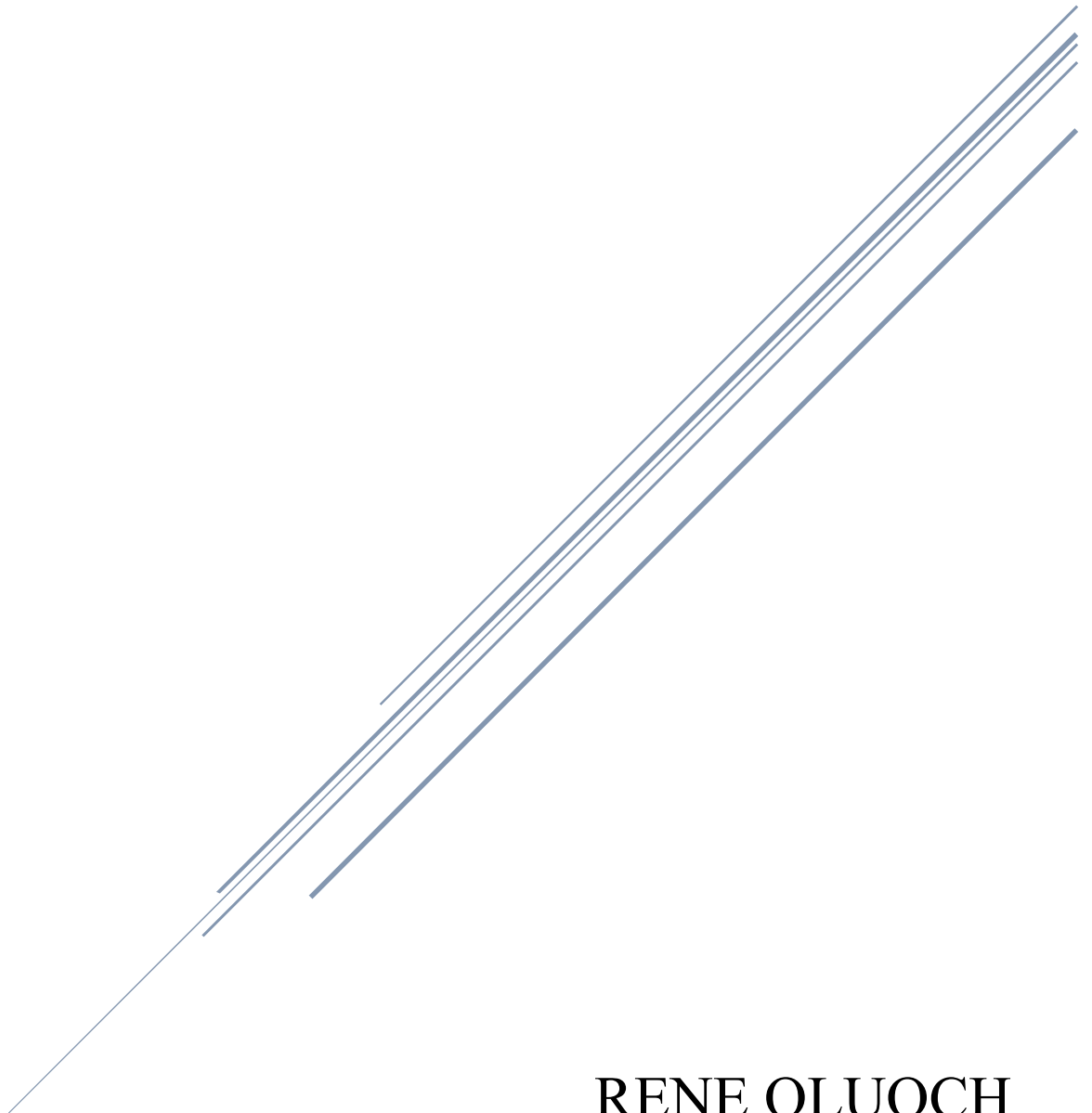


ASSIGNMENT 1

AZURE FIREWALL



RENE OLUOCH
CS-CNS09-25030

Table of Contents

Introduction.....	2
Task 1: Using a template to deploy the lab environment.....	2
Task 2: Deploying the Azure Firewall	5
Task 3: Creating a Default Route.....	9
Task 4: Creating and Applying an Application Rule	15
Task 5: Configuring a network rule	18
Task 6: Configuring the virtual machine DNS servers	21
Task 7: Testing the firewall	23
Conclusion	29

Introduction

In this assignment, I explored the practical deployment and configuration of Azure Firewall, a key security service in Microsoft Azure designed to control and monitor inbound and outbound traffic. Using the official Microsoft Learn lab resources and the provided template.json ARM template, I worked through the process of provisioning the necessary network infrastructure, deploying the firewall, defining custom routes, configuring firewall application and network rules, adjusting DNS settings for virtual machines, and verifying secure connectivity across the virtual network. This lab exercise deepened my understanding of how Azure Firewall enforces traffic policies and enhances overall cloud security by acting as a scalable, stateful packet inspection service for protecting resources within an Azure environment.

Task 1: Using a template to deploy the lab environment

To start this lab, I first signed in to the Azure portal by navigating to <https://portal.azure.com/> and logging in using my account which had sufficient permissions. Once inside the portal, I used the top search bar, labeled Search resources, services, and docs, to locate the Deploy a custom template option. After typing that phrase and pressing Enter, I was redirected to the Custom deployment blade.

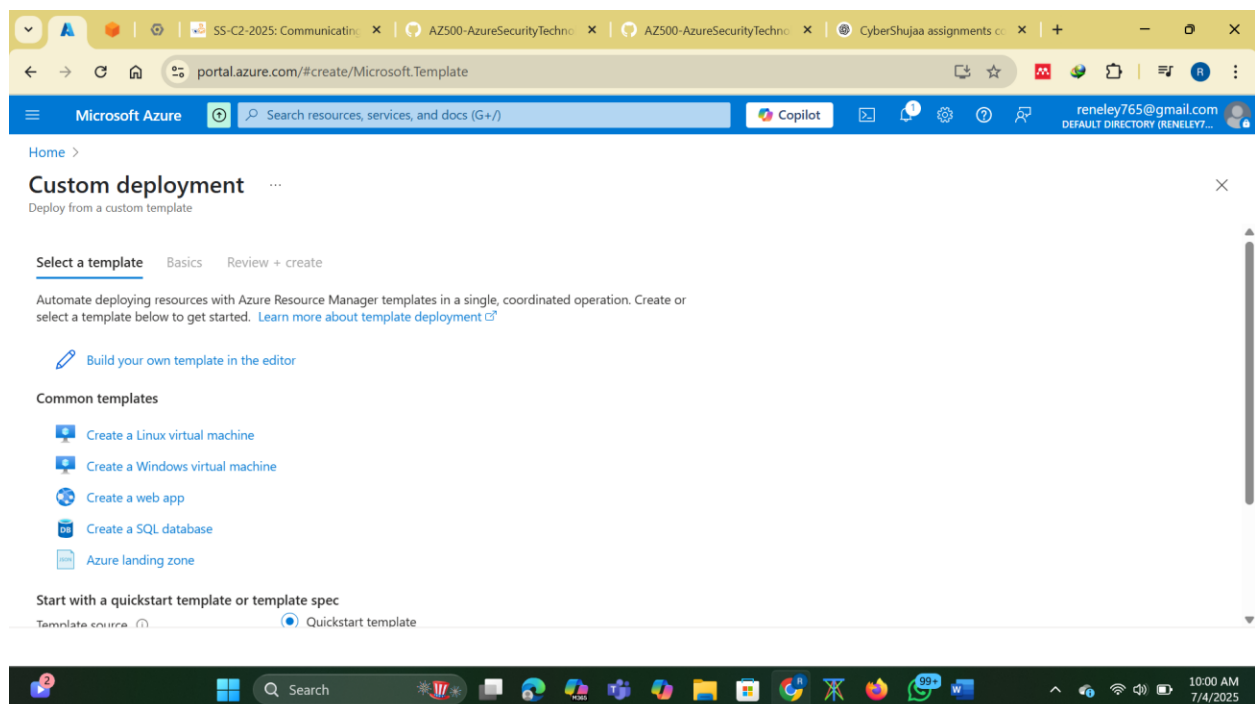


Fig 1.0 Custom deployment blade

On the Custom deployment page, I selected Build your own template in the editor, which opened the Edit template blade. There, I clicked Load file, navigated through my local file explorer to find the provided ARM template located at \\Allfiles\\Labs\\08\\template.json, and uploaded it. I carefully reviewed the contents of this JSON template to ensure that it would deploy a Windows Server 2016 Datacenter virtual machine as expected. Satisfied with the contents, I clicked Save to confirm the template.

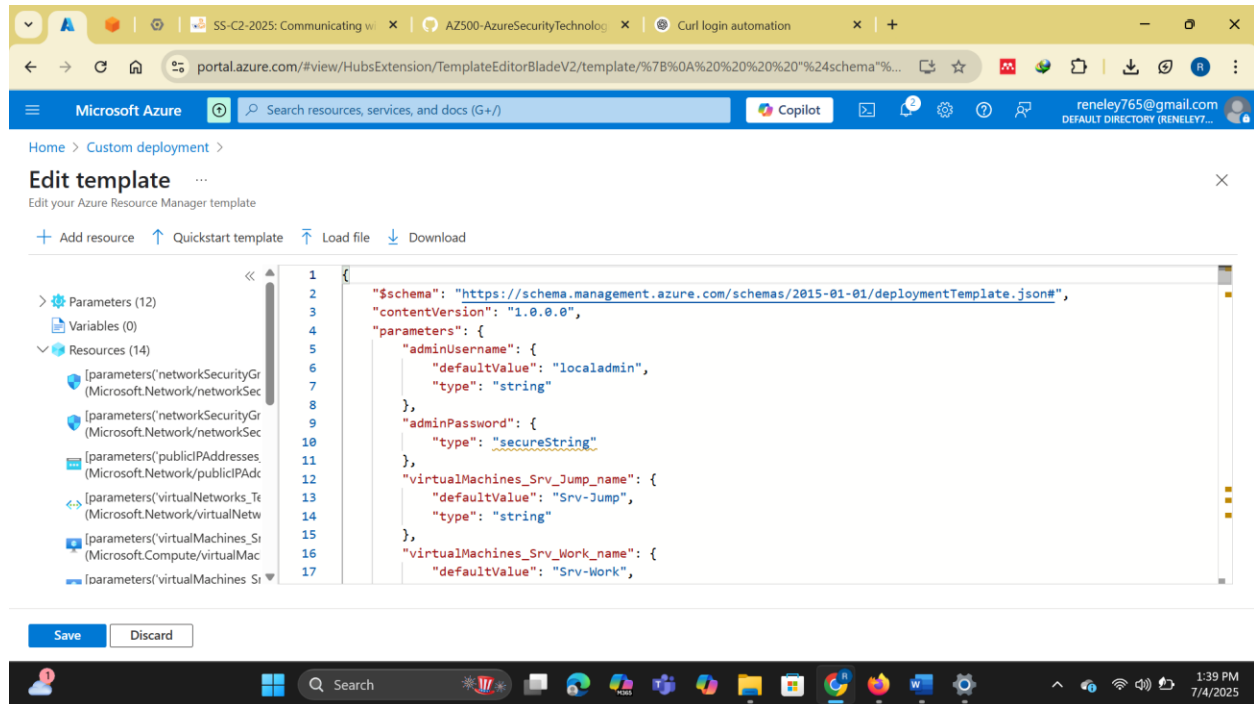


Fig 1.1 Loading json file and saving

Returning to the Custom deployment blade, I then filled out the necessary deployment parameters. I verified that the correct Subscription was selected, then created a new Resource Group named AZ500LAB08 to logically contain all resources for this lab. For the Location, I selected East US as specified in the lab instructions. Next, I defined a secure adminPassword for the virtual machine's administrator account(R3N3@LUOCH765), making sure to store it securely for later use when accessing the deployed virtual machines.

Microsoft Azure portal showing the 'Custom deployment' configuration page. The page includes the following fields and values:

- Subscription:** Azure subscription 1
- Resource group:** (New) AZ500LAB08
- Region:** East US
- Admin Username:** localadmin
- Admin Password:** [Masked]
- Virtual Machines_Srv_Jump_name:** Srv-Jump
- Virtual Machines_Srv_Work_name:** Srv-Work

Navigation buttons at the bottom: Previous, Next, Review + create.

Fig 1.2 Filling the deployment parameters

With these details filled in, I clicked Review + create to review the configuration one last time, ensuring that all fields matched the lab requirements and no accidental misconfigurations were present. Finally, I clicked Create to begin the deployment. I monitored the deployment progress in the Notifications pane until I saw confirmation that the deployment had completed successfully — this took approximately two minutes. This concluded the initial setup of the virtual network and the two virtual machines that I would later use for testing the Azure Firewall.

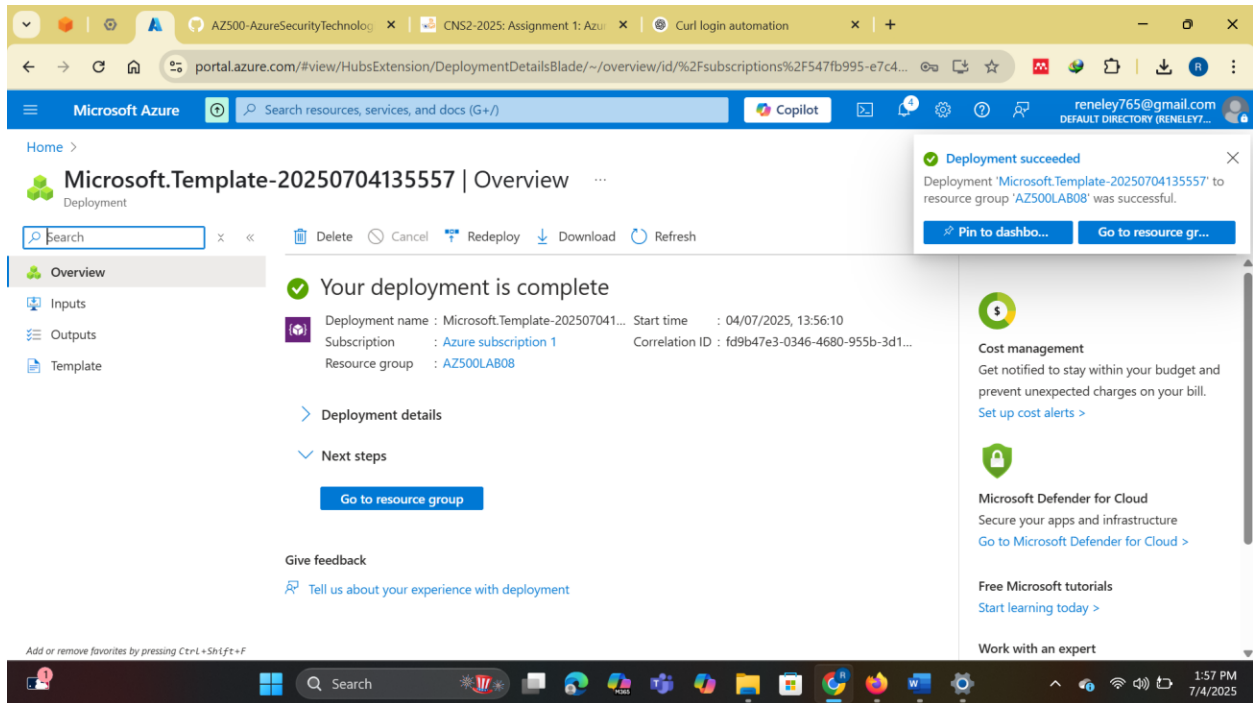


Fig 1.3 Complete deployment of lab environment

Task 2: Deploying the Azure Firewall

After successfully setting up the foundational environment in Task 1, I proceeded to deploy the Azure Firewall. I returned to the Azure portal's main dashboard and used the Search resources, services, and docs bar at the top of the page to locate the Firewalls blade by typing Firewalls and pressing Enter. Once the Firewalls blade opened, I clicked the + Create button to start the deployment wizard for a new firewall instance.

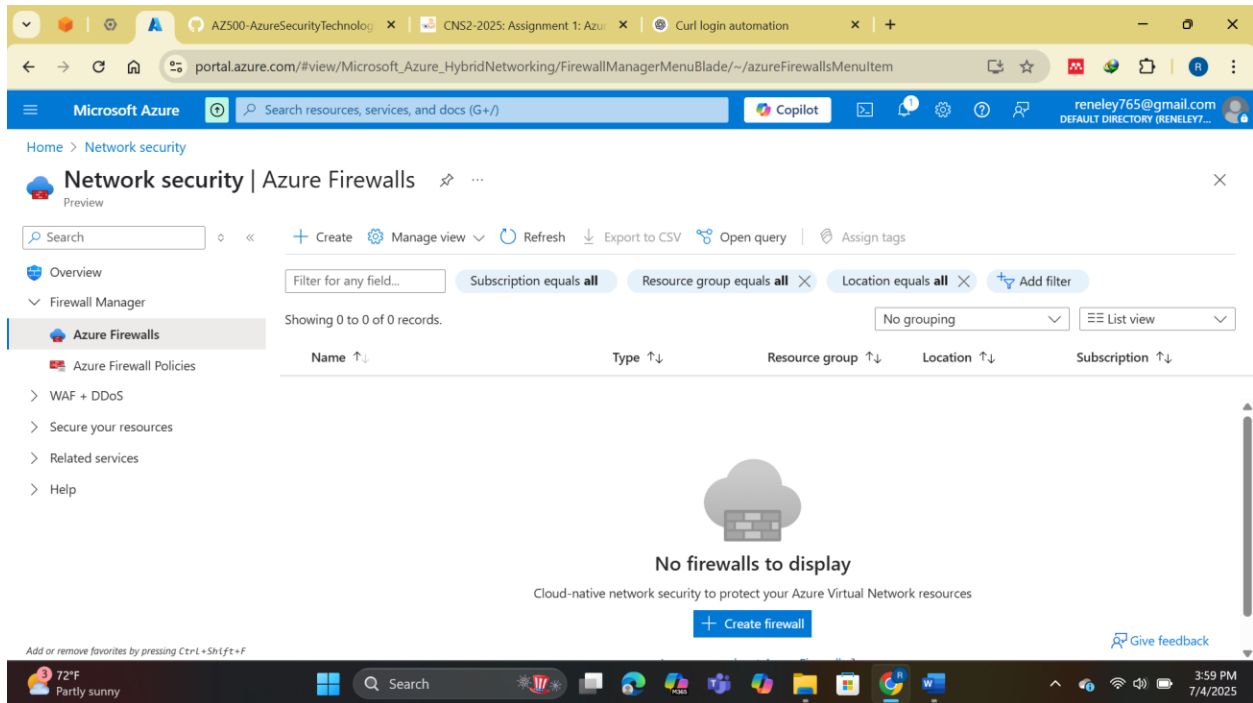


Fig 1.4 Firewall blade

On the Basics tab of the Create a firewall page, I began by selecting the same Resource Group I had created earlier — AZ500LAB08 — ensuring that all related resources would remain logically grouped and easy to manage. For the Name of the firewall, I entered Test-FW01, following the naming convention provided in the lab scenario. I confirmed the Region was set to East US, keeping it consistent with all other deployed resources.

Next, I selected the Standard SKU for the firewall to meet the lab's requirement for a baseline level of Azure Firewall functionality. For Firewall management, I chose the option Use Firewall rules (classic) to manage this firewall using classic rule collections instead of newer policy-based management, aligning with the lab instructions. For the network integration, I clicked Use existing under the Choose a virtual network section and then selected the previously created virtual network named Test-FW-VN from the drop-down list. I made sure to deselect the Enable Firewall Management NIC option since the lab does not require a dedicated management interface for this firewall deployment. To allocate a public IP address for the firewall, I clicked Add new, provided the name TEST-FW-PIP, and confirmed by clicking OK. This ensured that the firewall would have a public-facing IP for necessary external connectivity.

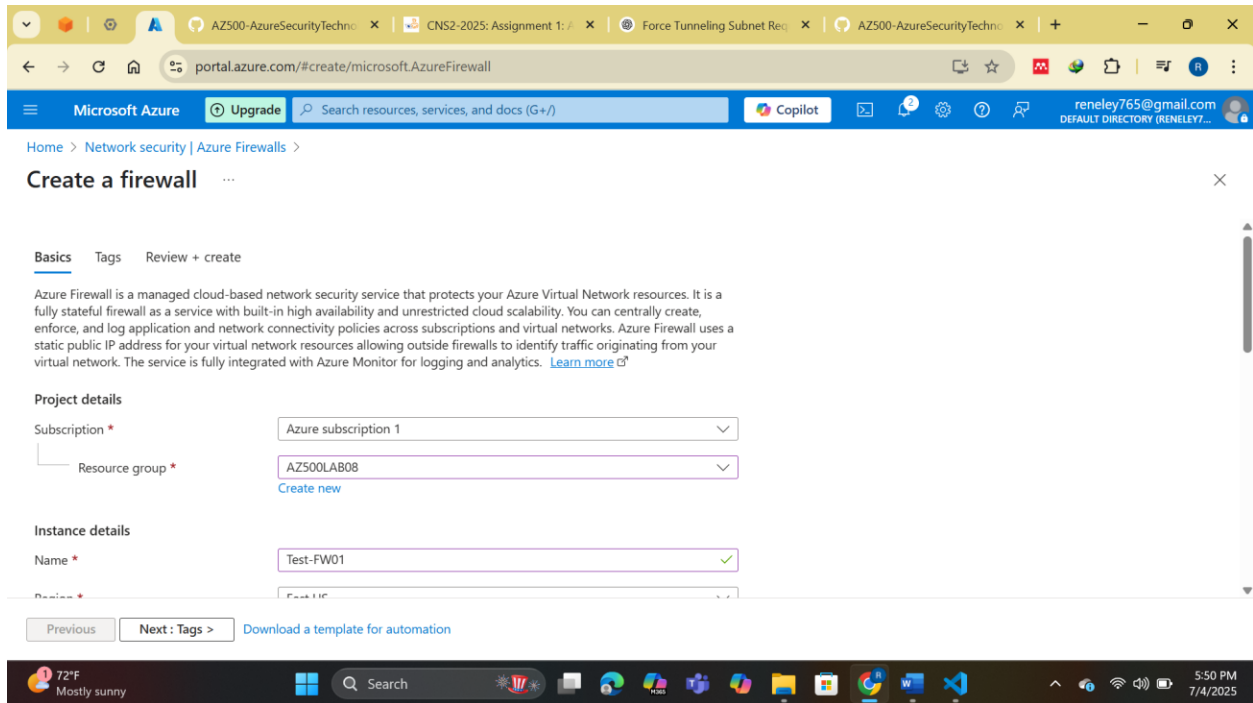


Fig 1.5 Filling in the required parameters

After verifying all the specified configurations — resource group, name, region, SKU, management mode, virtual network selection, and public IP setup — I clicked Review + create to validate the settings, and then clicked Create to launch the deployment of the firewall resource.

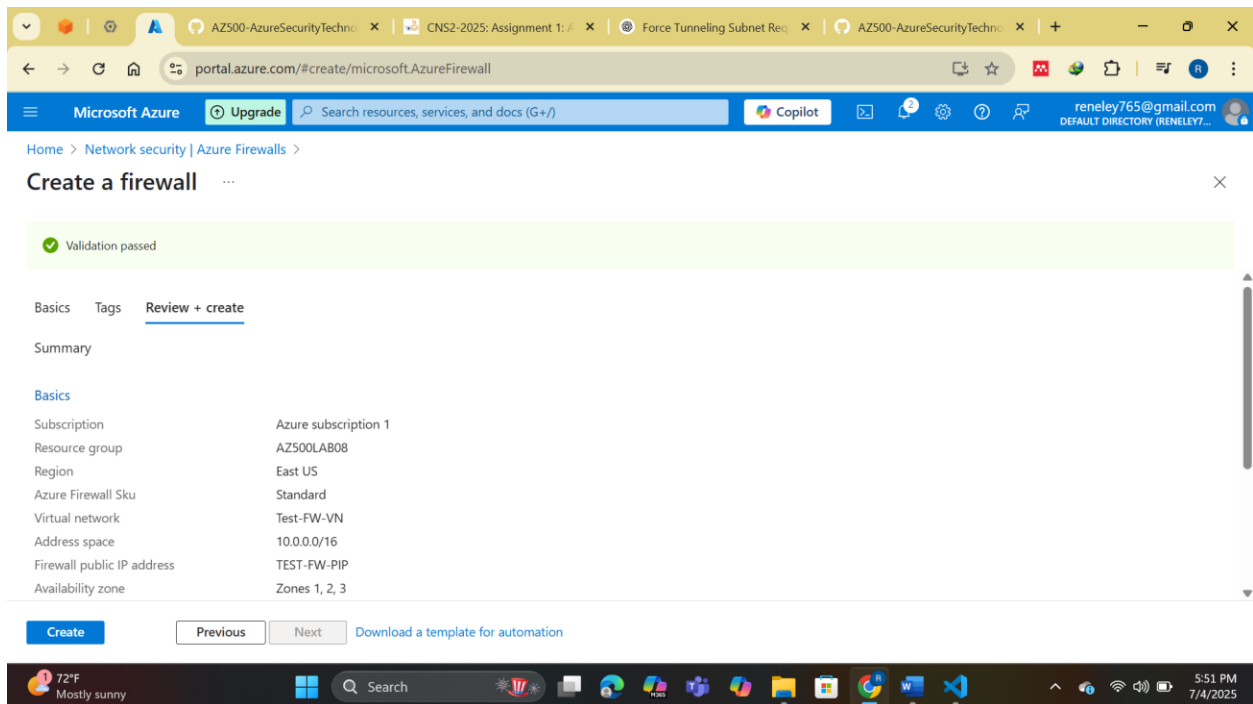


Fig 1.6 Reviewing and creating the firewall

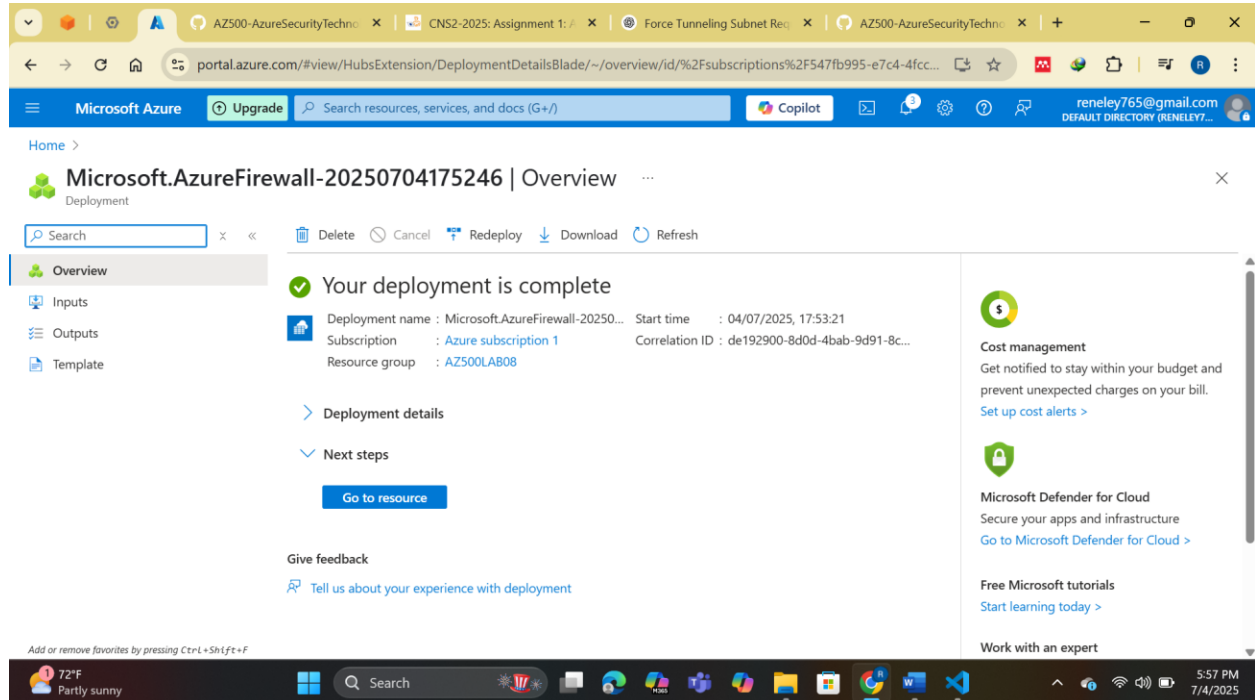
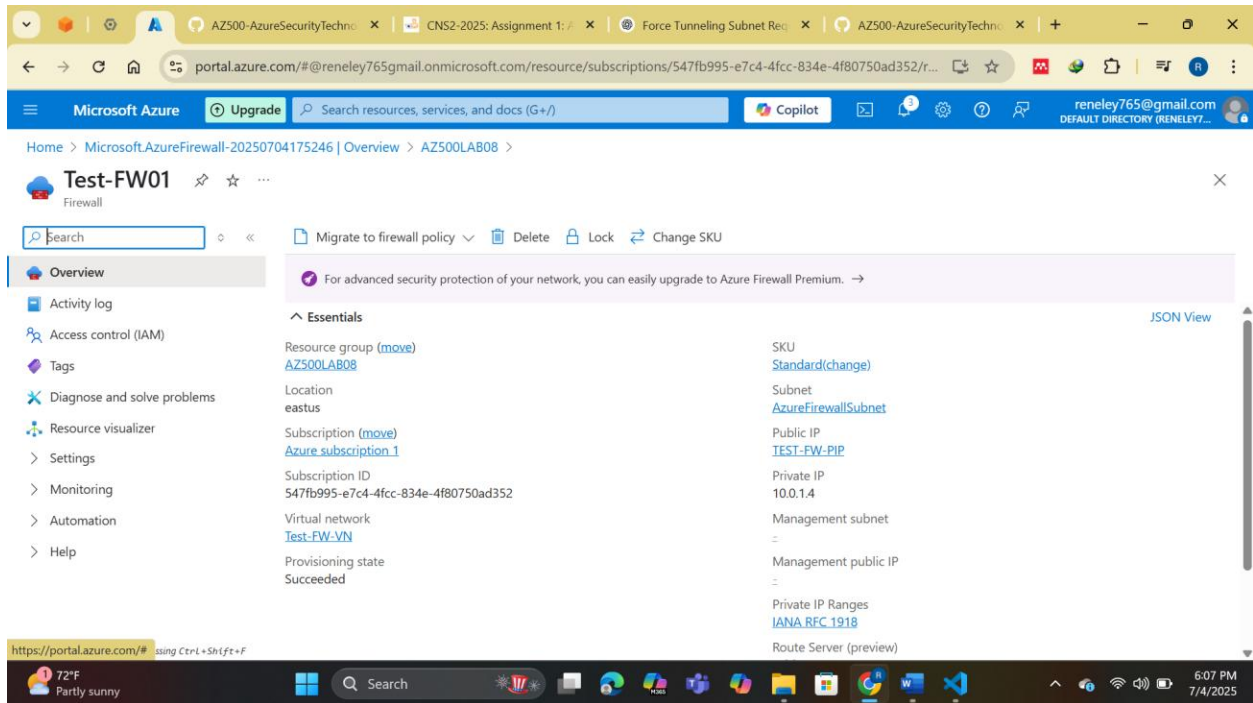


Fig 1.7 Completed deployment

Once the firewall resource was provisioned, I navigated back to Resource groups, located AZ500LAB08, and clicked into it to review the list of deployed resources. Within this list, I located Test-FW01, clicked on it, and noted its Private IP address, which I would need for routing configuration in the next task.



Fih 1.8 Private ip on Test-FW01

Task 3: Creating a Default Route

After confirming that the Test-FW01 firewall was deployed and noting its Private IP address, I moved on to configure the routing so that all outbound traffic from the workload subnet would be forced through the firewall, as required by the lab scenario.

To start, I went back to the Azure portal's main dashboard. Using the Search resources, services, and docs bar, I typed Route tables and pressed Enter to open the Route tables blade. On this blade, I clicked the + Create button to launch the Create route table wizard.

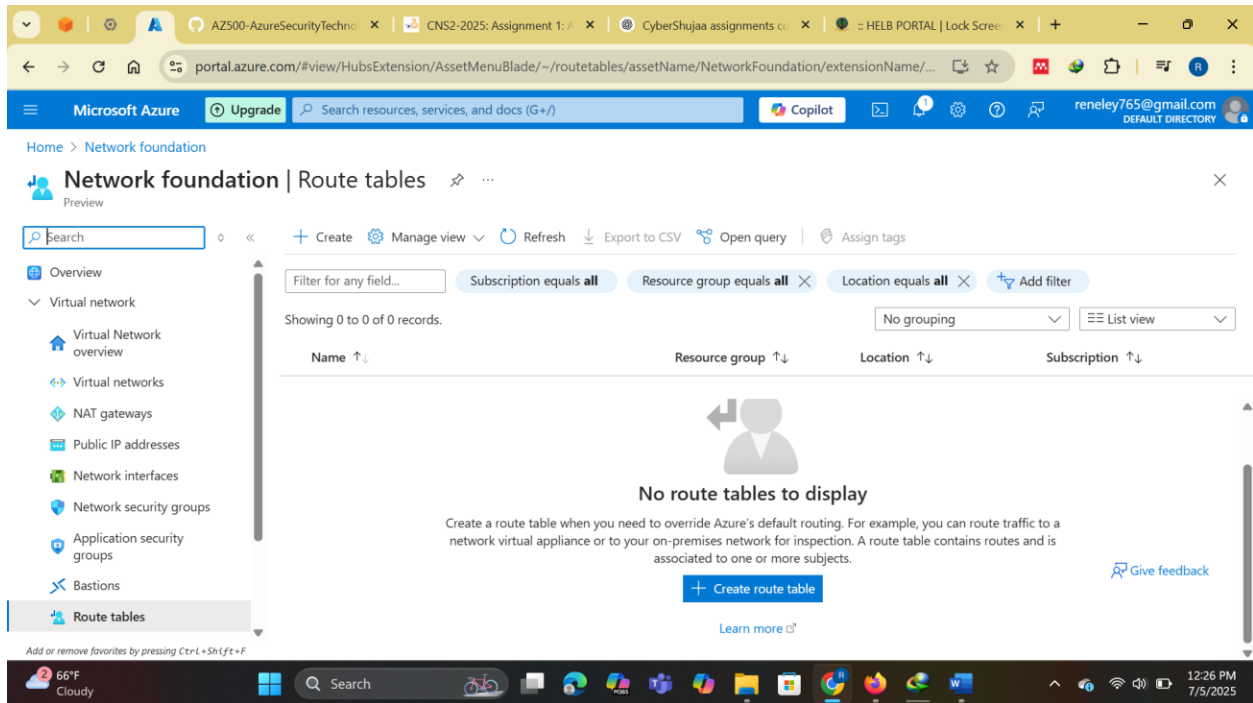


Fig 1.9 Route table blade

On the Create route table page, I set the Resource group to AZ500LAB08 to ensure the new route table stayed organized with the rest of the lab resources. For the Region, I selected East US, which matched the location of my virtual network and firewall. I named the route table Firewall-route to clearly indicate its purpose — routing outbound traffic through the firewall.

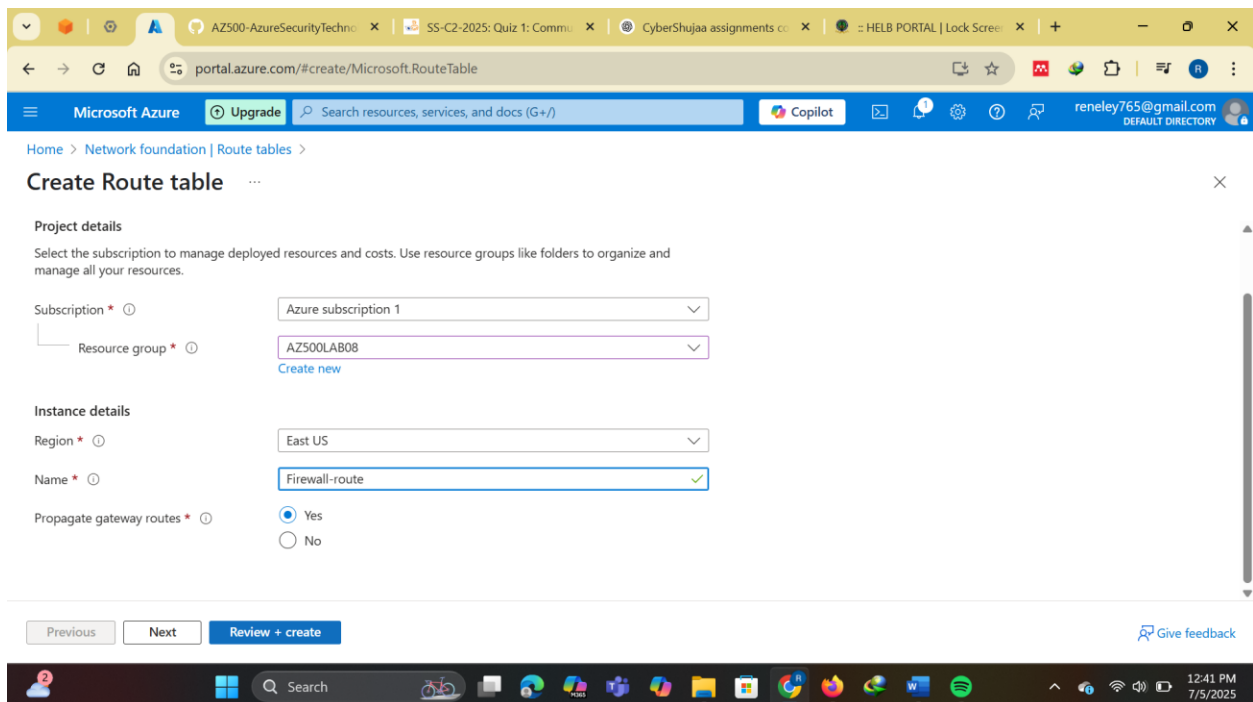


Fig 2.0 Creating route table and Filling in the required details

With those settings confirmed, I clicked Review + create and then Create to deploy the new route table. When the deployment completed, I refreshed the Route tables blade and located the newly created Firewall-route entry in the list. I clicked on it to open its management blade.

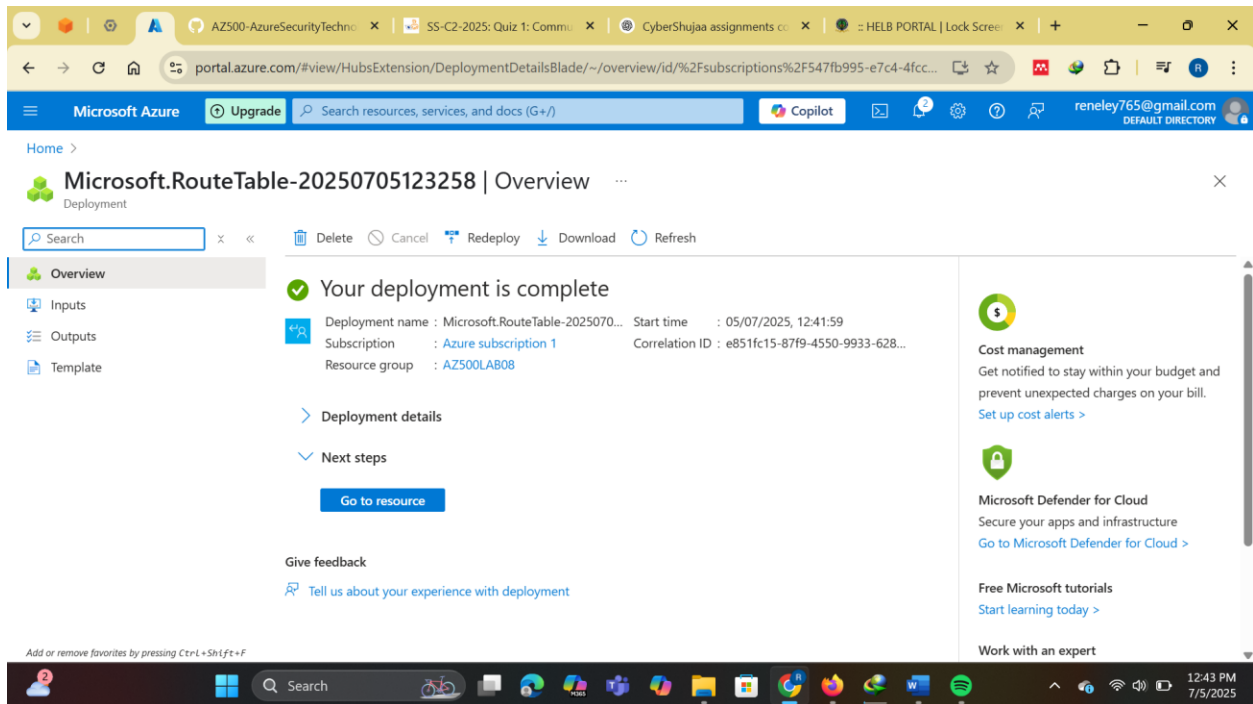


Fig 2.1 Complete deployment of the route table

Within the Firewall-route blade, under the Settings section, I selected Subnets to associate this route table with the appropriate subnet. On the Subnets page, I clicked + Associate, which opened the Associate subnet dialog. In the association form, I made sure to select Test-FW-VN as the Virtual network — this is the virtual network that contains both the workload subnet and the firewall. For the Subnet, I specifically chose Workload-SN, the workload subnet that needs its outbound traffic inspected and filtered by the Azure Firewall. After verifying that the correct subnet was selected, I clicked OK to bind the route table to Workload-SN.

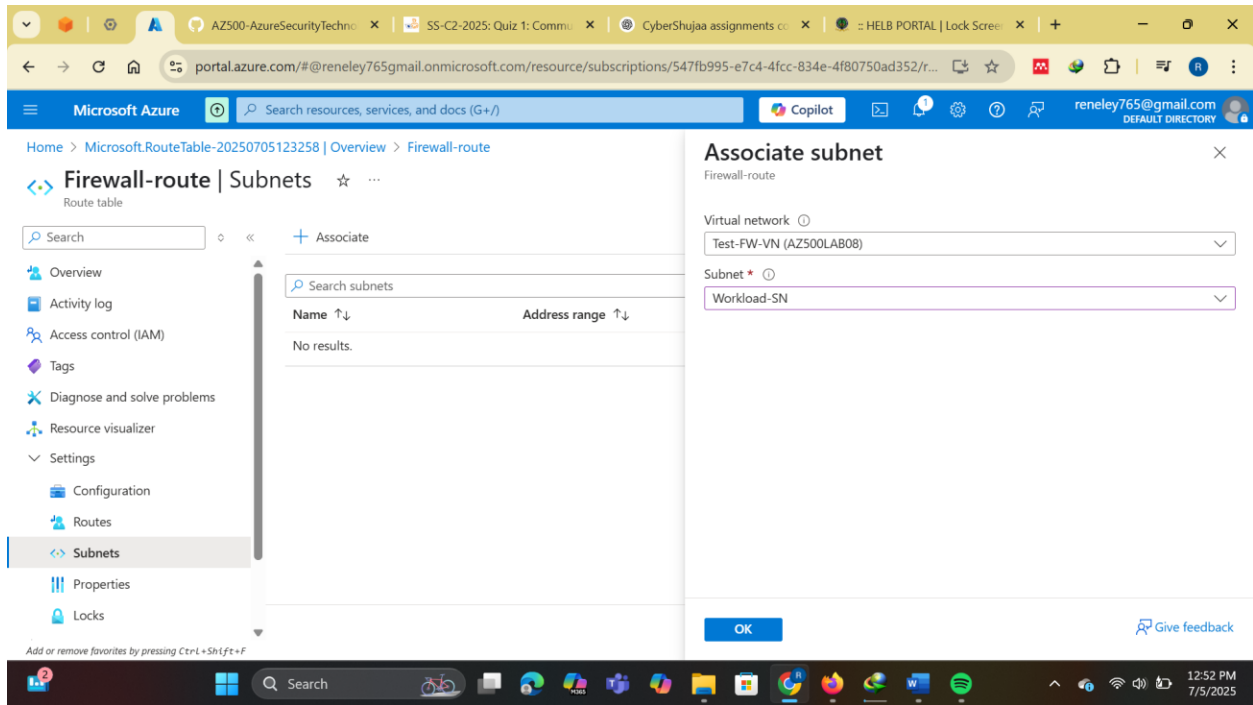


Fig 2.2 Fillin in required details for the firewall route

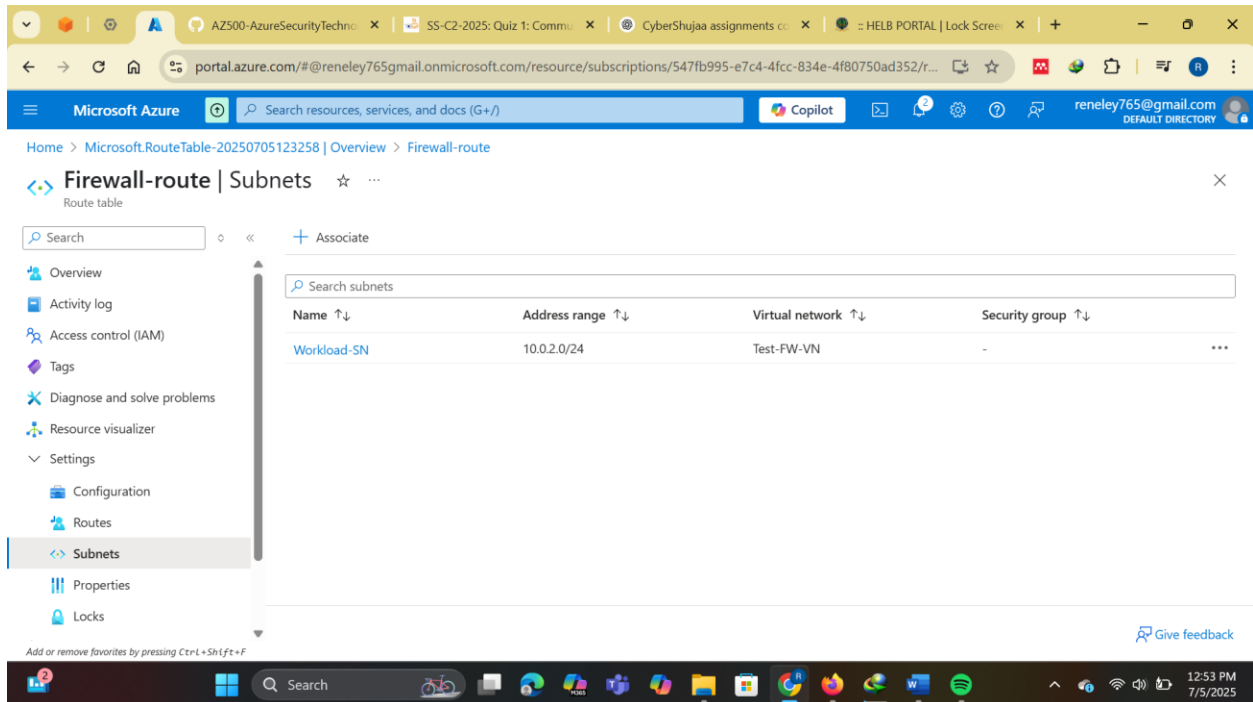


Fig 2.3 Completed association of firewall route subnets

With the association complete, I went back to the Firewall-route blade and, still under the Settings section, clicked Routes to define the custom routing rule that would direct outbound traffic through the firewall. On the Routes page, I clicked + Add to create a new route.

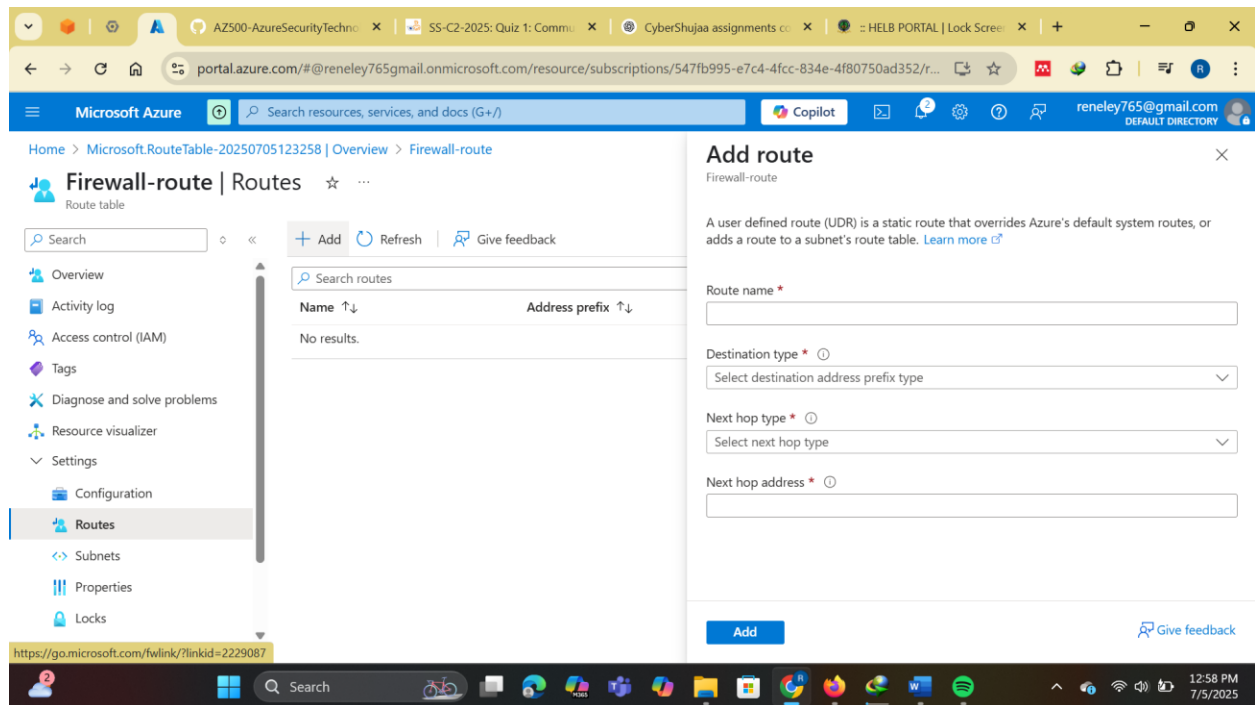


Fig 2.4 Adding firewall route

In the Add route dialog, I specified the following: Route name: FW-DG to indicate that this is a default gateway route for the firewall. Destination type: IP Address. Destination IP addresses/CIDR ranges: 0.0.0.0/0 — this CIDR means *all* outbound traffic. Next hop type: Virtual appliance — which in this case refers to the Azure Firewall acting as a virtual appliance. Next hop address: I pasted the Private IP address of Test-FW01 that I noted at the end of Task 2.

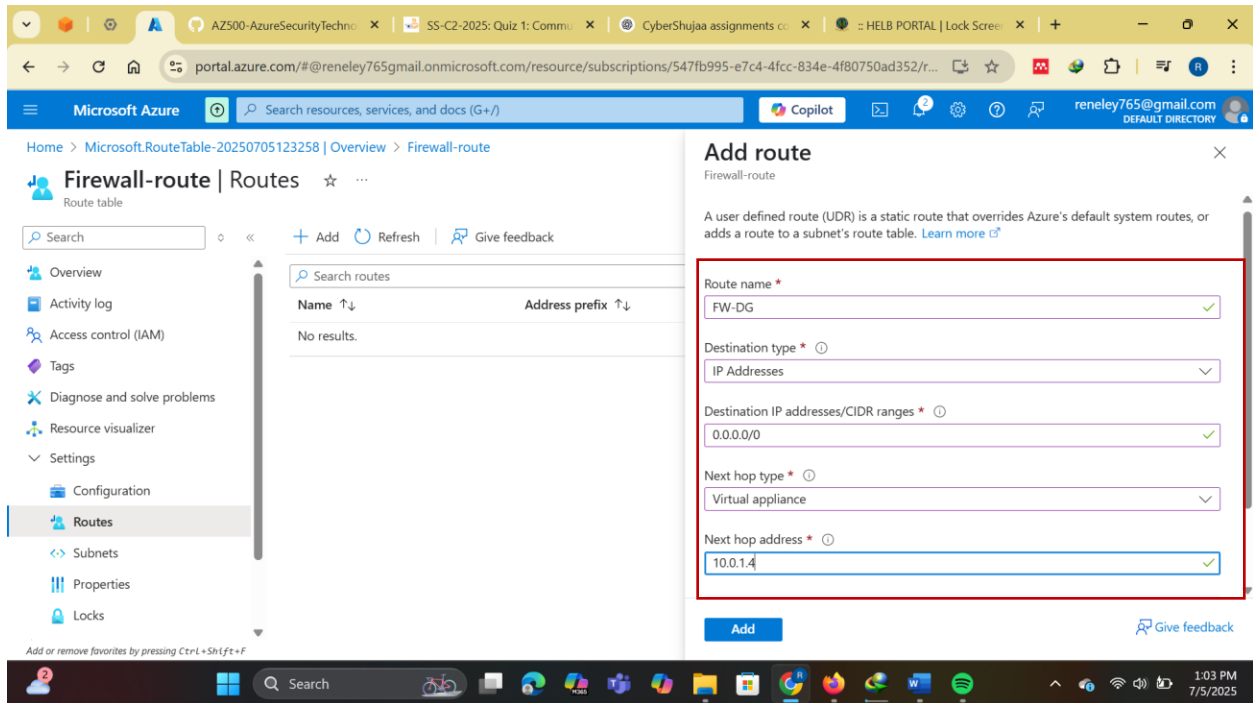


Fig 2.5 Filling in the required fields for the route

Once all fields were verified, I clicked Add to finalize and apply the route.

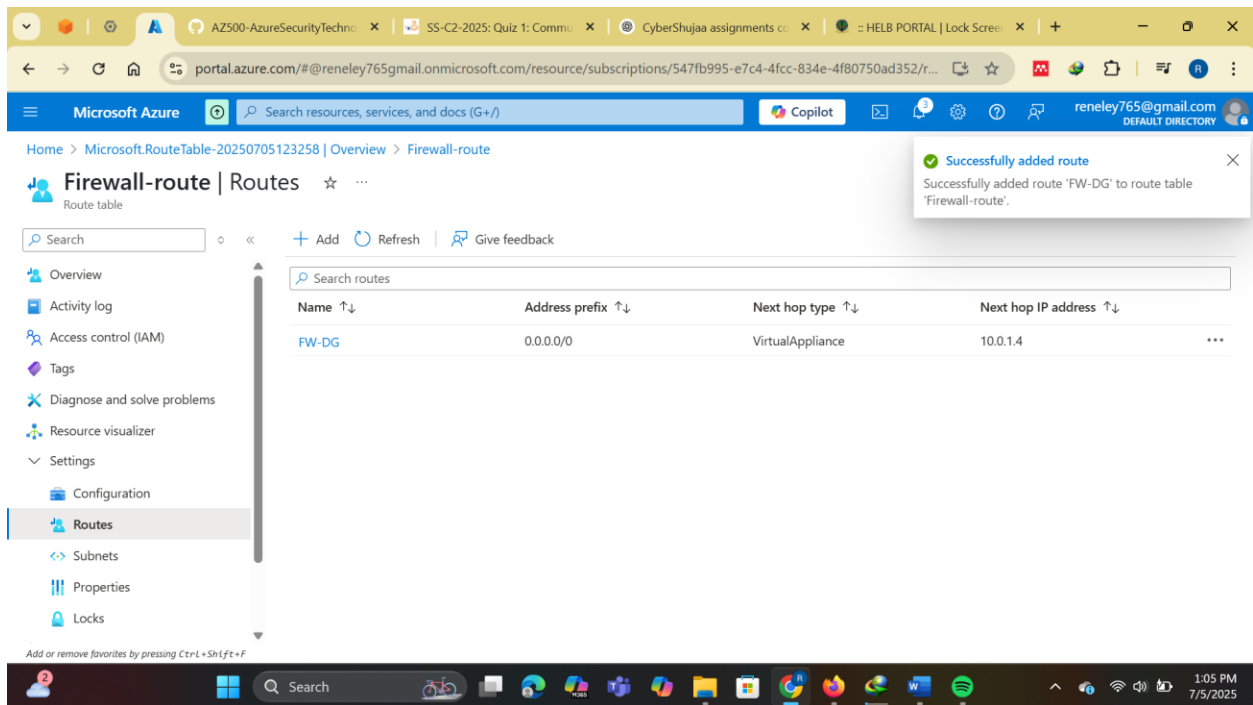


Fig 2.6 Successfully adding the route

Task 4: Creating and Applying an Application Rule

After successfully configuring the default route that forced all outbound traffic from the Workload subnet through the Azure Firewall, I proceeded to create an application rule to precisely control which websites could be accessed. To do this, I navigated back to the Azure portal and opened the **Test-FW01** firewall resource that I had deployed earlier. On the firewall's management blade, I located the **Rules** section under **Settings**, then selected **Rules (classic)** and moved to the **Application rule collection** tab, which is specifically meant for managing rules that filter outbound web traffic by domain names and protocols.

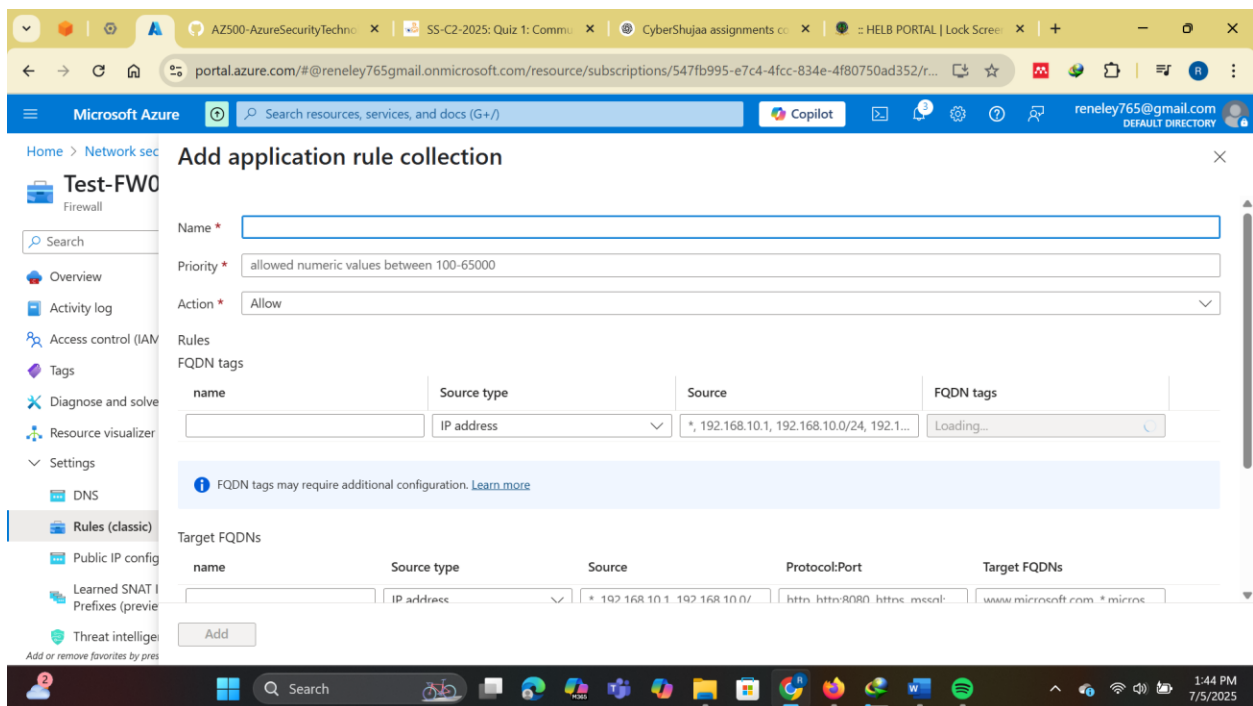


Fig 2.7 Adding application rule collection

I initiated the creation of a new application rule collection by clicking + **Add application rule collection**, which opened a configuration pane where I specified the collection's name as **Allow-Web**, assigned it a priority of **200** to ensure it would be evaluated before any lower-priority rules, and set the action to **Allow**, since the goal was to explicitly permit certain traffic.

Within this same pane, I defined the actual rule by giving it the name **Allow-MS**, setting the source type to IP Addresses so that I could restrict the rule to the **Workload-SN** subnet's range, which is **10.0.2.0/24**, ensuring that only traffic originating from this range would be matched. For the protocol, I selected **HTTP** to allow standard web requests on TCP port 80, and in the target FQDNs field, I entered www.microsoft.com to allow connections solely to this domain.

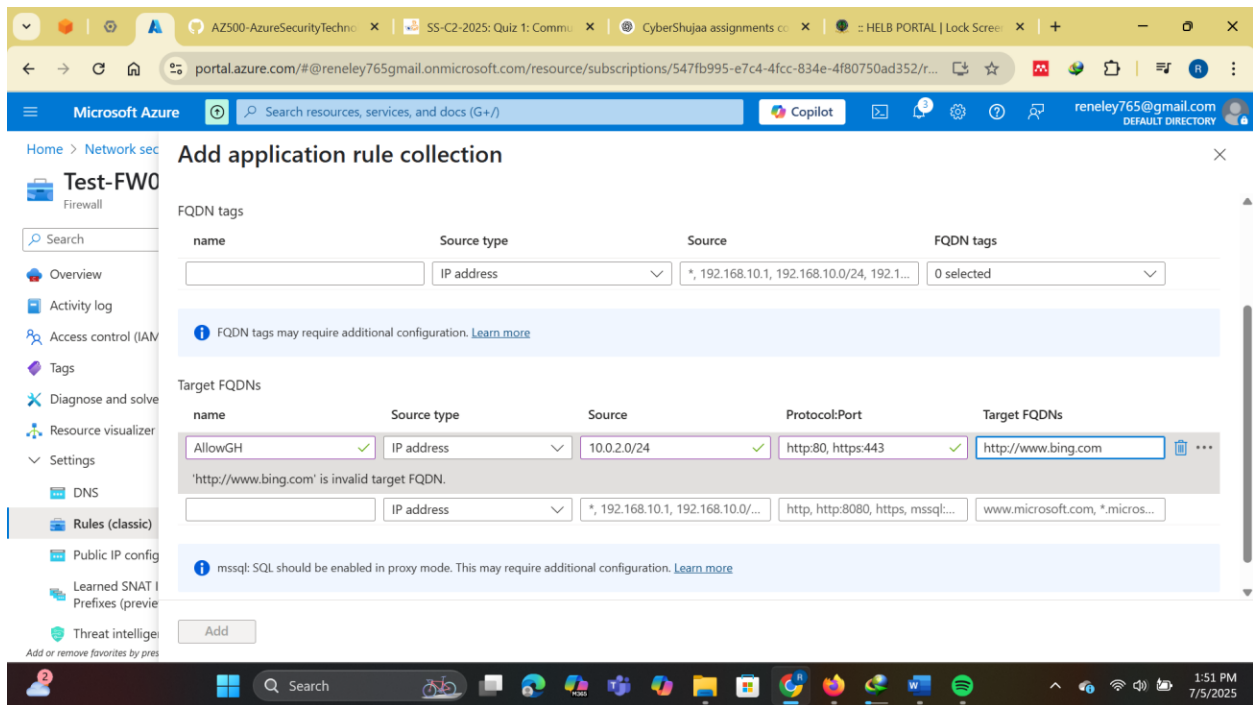


Fig 2.8 Filling in the required details the application rule

After verifying every field carefully to confirm that the source address, protocol, and allowed domain were correctly defined to match the security requirement, I clicked **Add** to save and apply the rule collection.

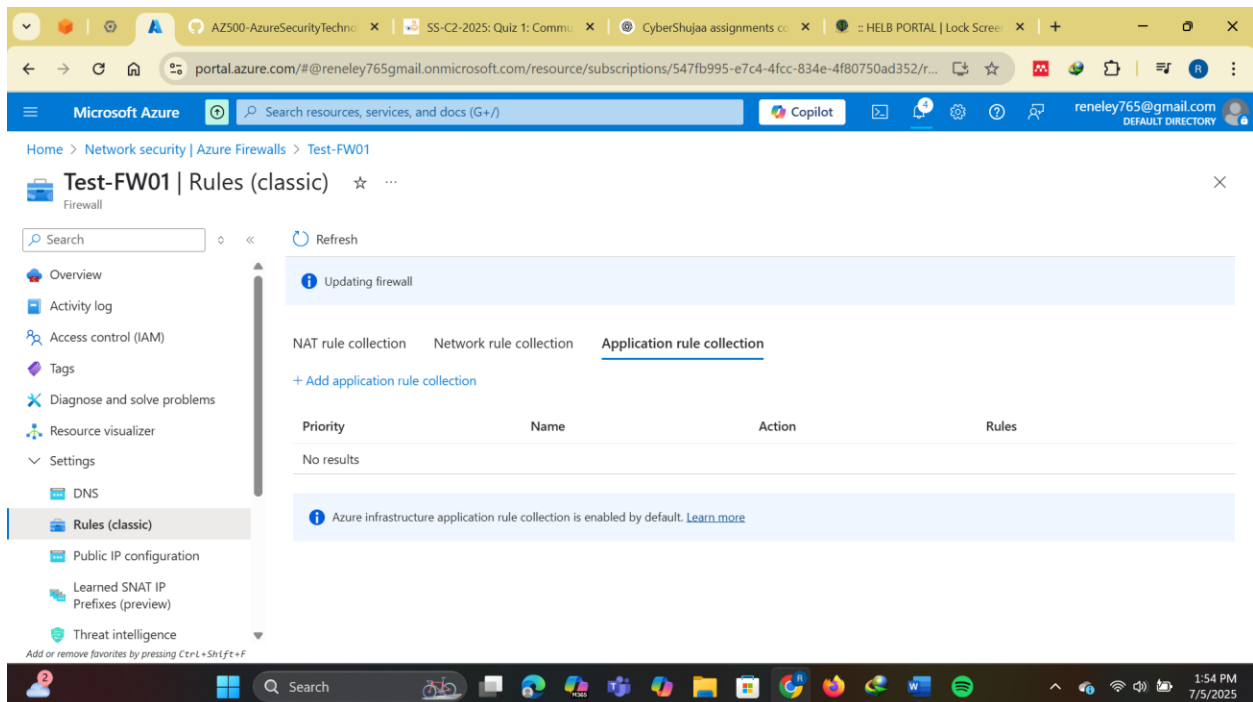


Fig 2.9 Updating firewall

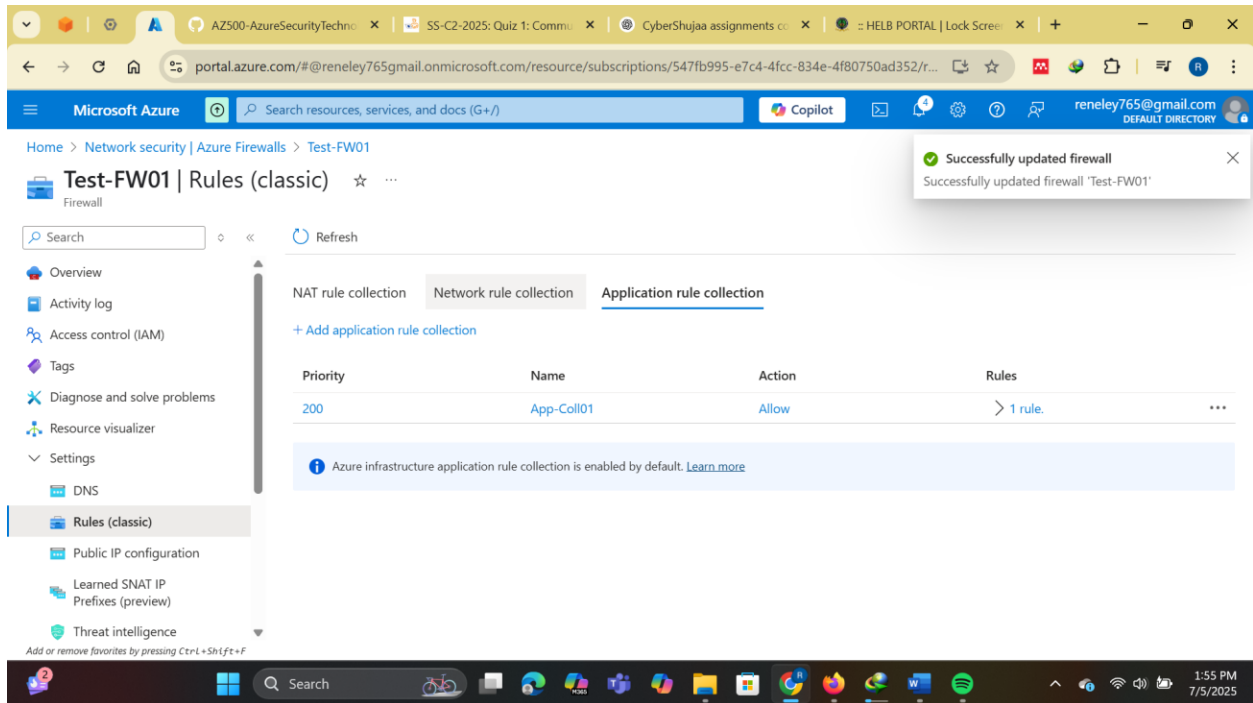


Fig 3.0 Successfully updating firewall

Once the rule was added, I reviewed the list of application rules on the firewall to ensure that **Allow-Web** appeared with the proper priority and that the single rule within it exactly matched the intended configuration. By finishing this step, I confirmed that outbound web traffic from the workload subnet was now restricted to HTTP requests only to www.microsoft.com, with all other domains implicitly blocked by the firewall's default deny behavior.

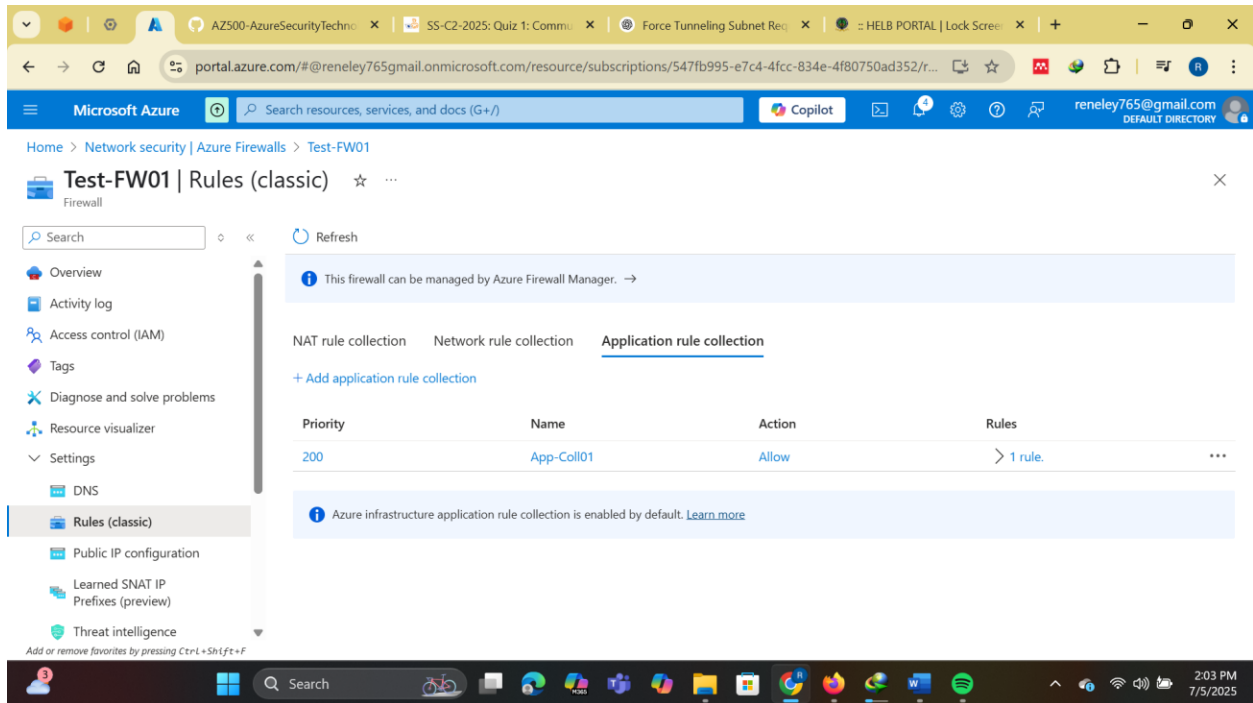


Fig 3.1 Reviewing list of configs to ensure proper configuration

Task 5: Configuring a network rule

I moved on to Task 5, which involved setting up a network rule to allow DNS resolution for outbound traffic from the workload subnet. With the Azure Firewall still open in the portal under the **Test-FW01** resource, I stayed within the **Rules (classic)** section but this time switched to the **Network rule collection** tab, which is designed to handle traffic rules that are based on IP addresses and ports rather than fully qualified domain names.

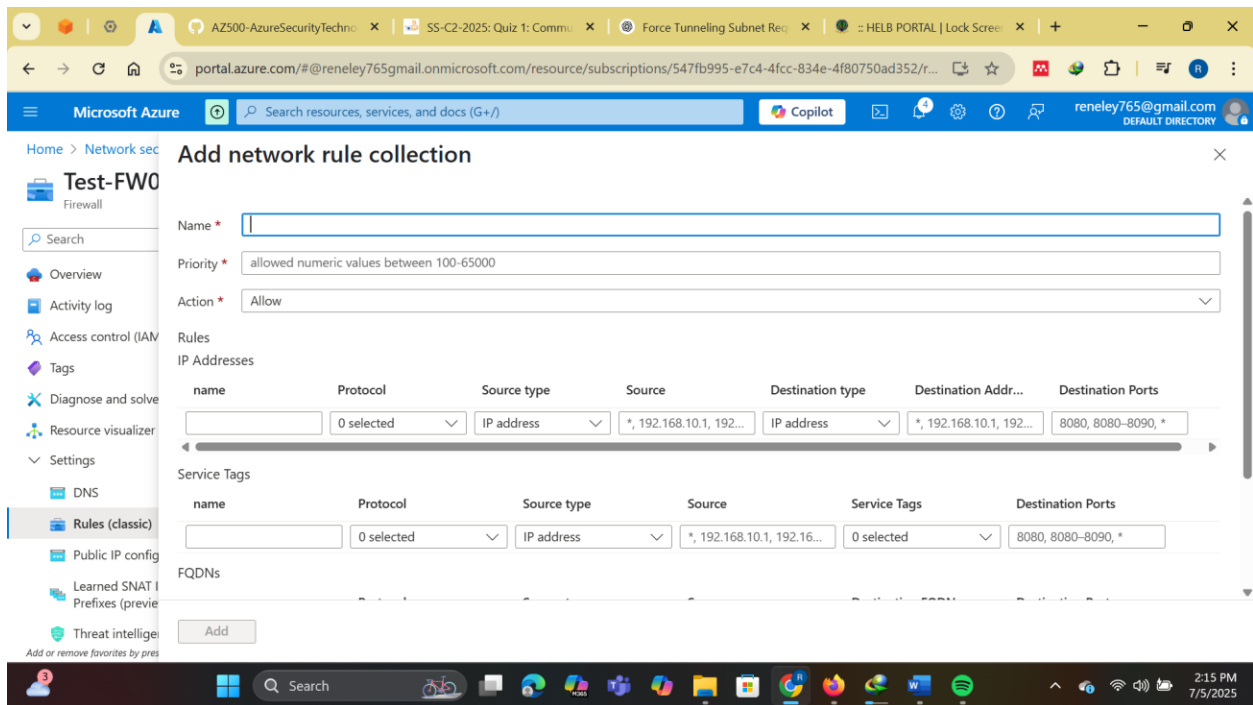


Fig 3.2 Adding network rule

To create this new network rule collection, I clicked on + **Add network rule collection**, which opened the configuration pane where I began by naming the collection **Net-Allow-DNS** to make its purpose clear for future auditing. I assigned it a priority of **200**, giving it high precedence to ensure that DNS queries are always permitted before any more restrictive rules could take effect, and set the action to **Allow** because the objective was to permit DNS lookups from the virtual machines in the workload subnet to reach external DNS servers. Within the same pane, I added the specific rule by naming it **Allow-DNS-UDP**, since DNS uses the UDP protocol by default for most queries. For the source type, I selected **IP address** and entered **10.0.2.0/24** to precisely limit the allowed traffic to the workload subnet's address space. I specified **UDP** as the protocol and entered the two well-known external DNS server IP addresses **209.244.0.3** and **209.244.0.4** as the destinations. To restrict the traffic even further, I defined the destination port as **53**, which is the standard port used for DNS queries.

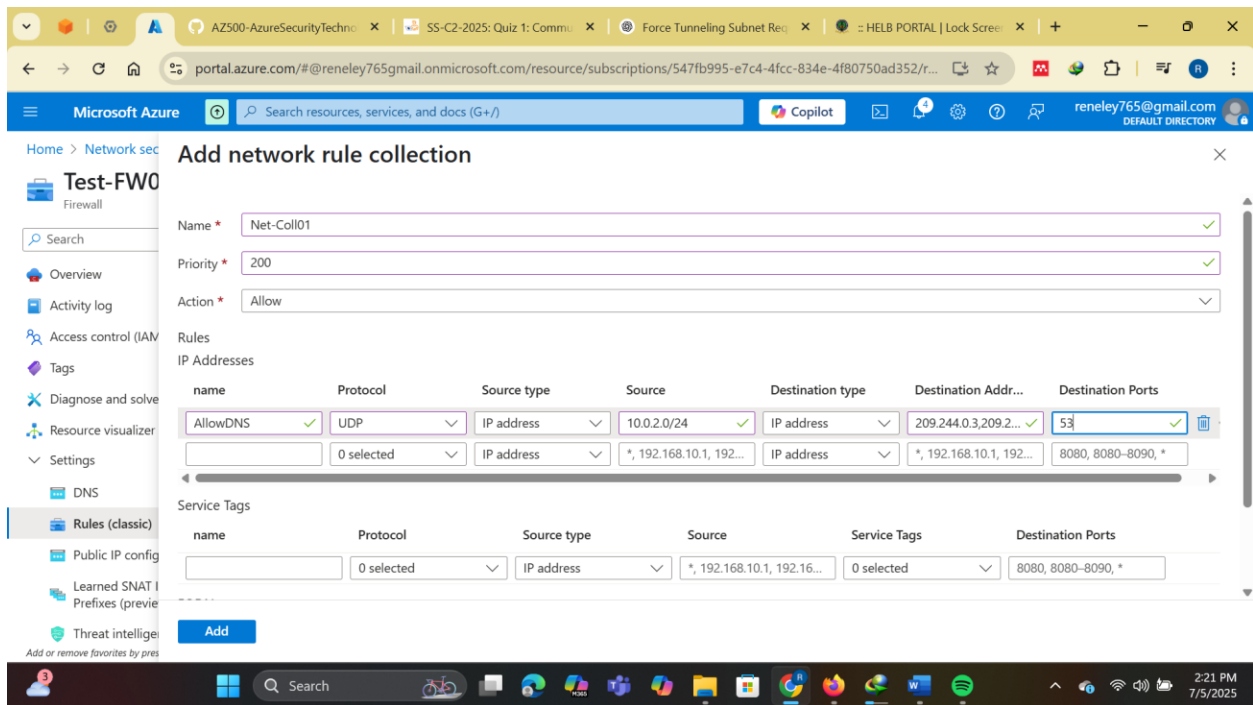


Fig 3.3 Filling in the required fields for the network rule

After reviewing every field to ensure that the source subnet, protocol, destination IPs, and port were correctly defined to only allow the necessary DNS traffic, I clicked **Add** to finalize the rule and include it in the **Net-Allow-DNS** network rule collection. Once the configuration was saved, I verified that the new network rule appeared in the list with the intended priority and action. This ensured that virtual machines within the workload subnet would now be able to resolve domain names using the specified external DNS servers, while still being fully controlled by the Azure Firewall's strict outbound filtering policies.

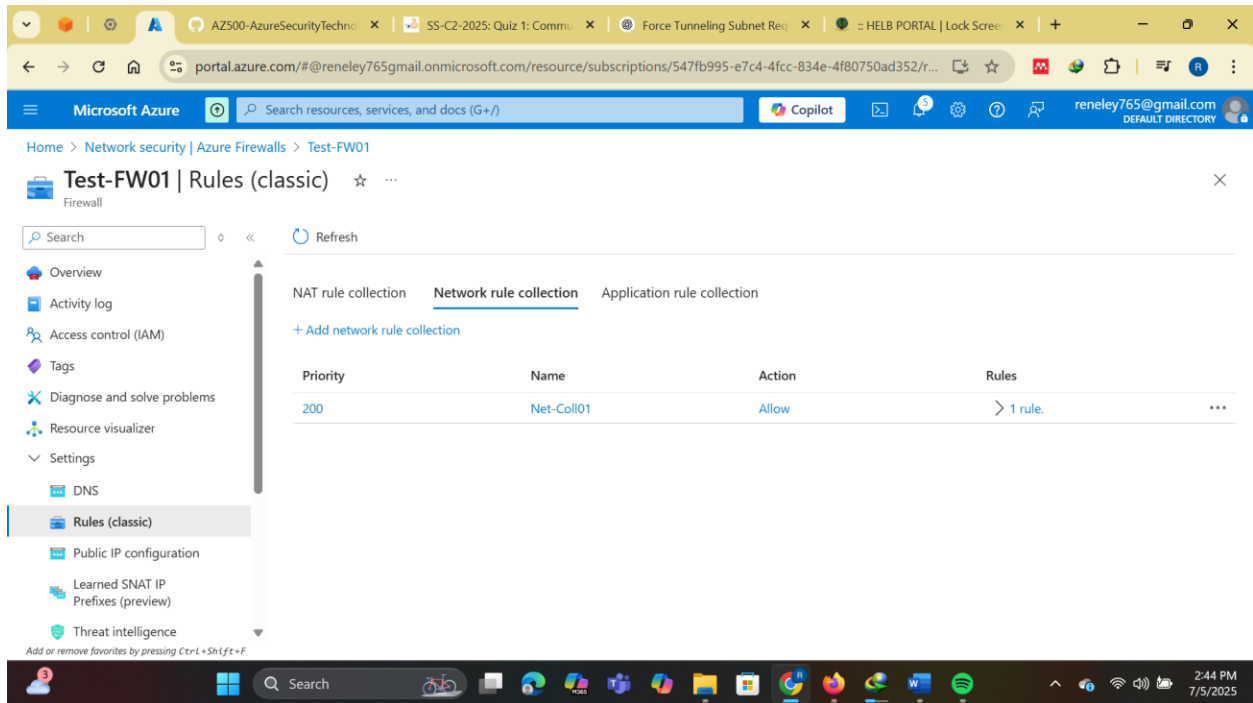


Fig 3.4 Successfully creating network rules

Task 6: Configuring the virtual machine DNS servers

Here I was required to configure the virtual machine's DNS servers so that it would actually use the public DNS servers I had just allowed through the firewall rules.

Remaining within the same **AZ500LAB08** resource group in the Azure portal, I navigated to the list of deployed resources and located the **Srv-Work** virtual machine, which is the workload VM sitting in the workload subnet. I selected the **Srv-Work** VM to open its overview blade, and from there, I went to the **Networking** section in the left-hand menu.

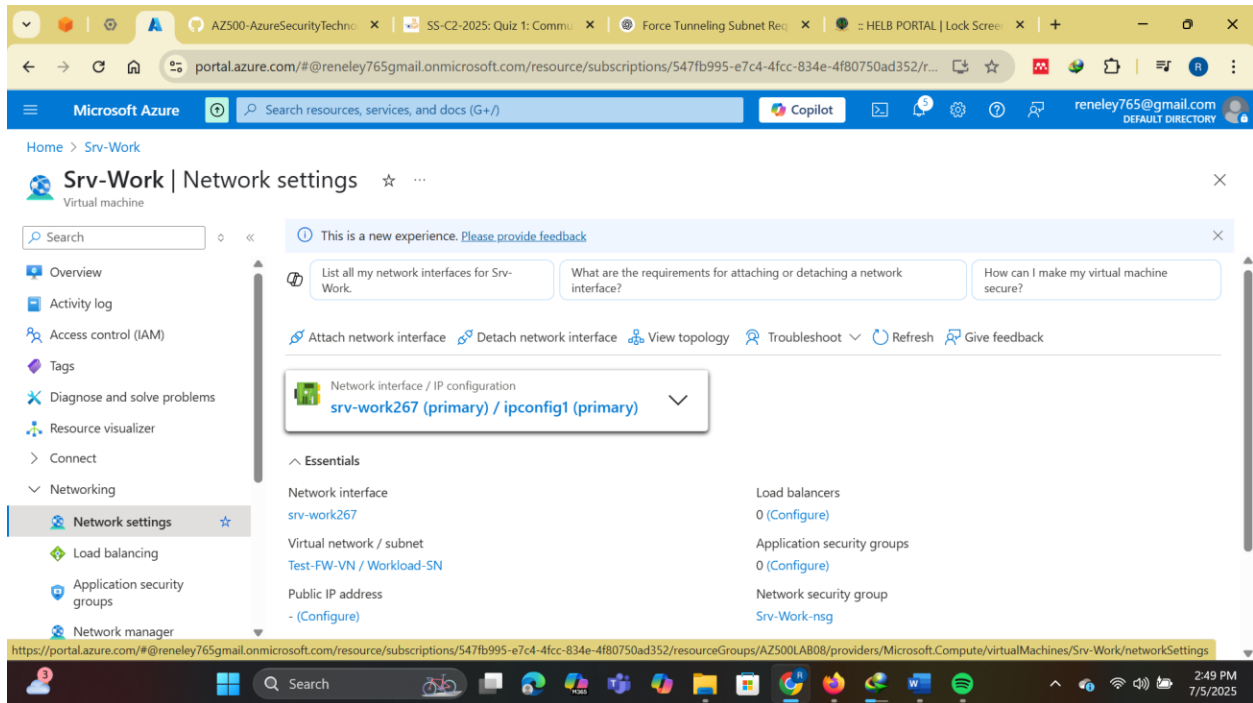


Fig 3.5 Srv-Work pane(Network Section)

On the networking settings page, I looked for the network interface that the VM is attached to — this is where DNS settings are configured at the NIC level. I clicked the link for the network interface name, which opened the NIC’s dedicated blade.

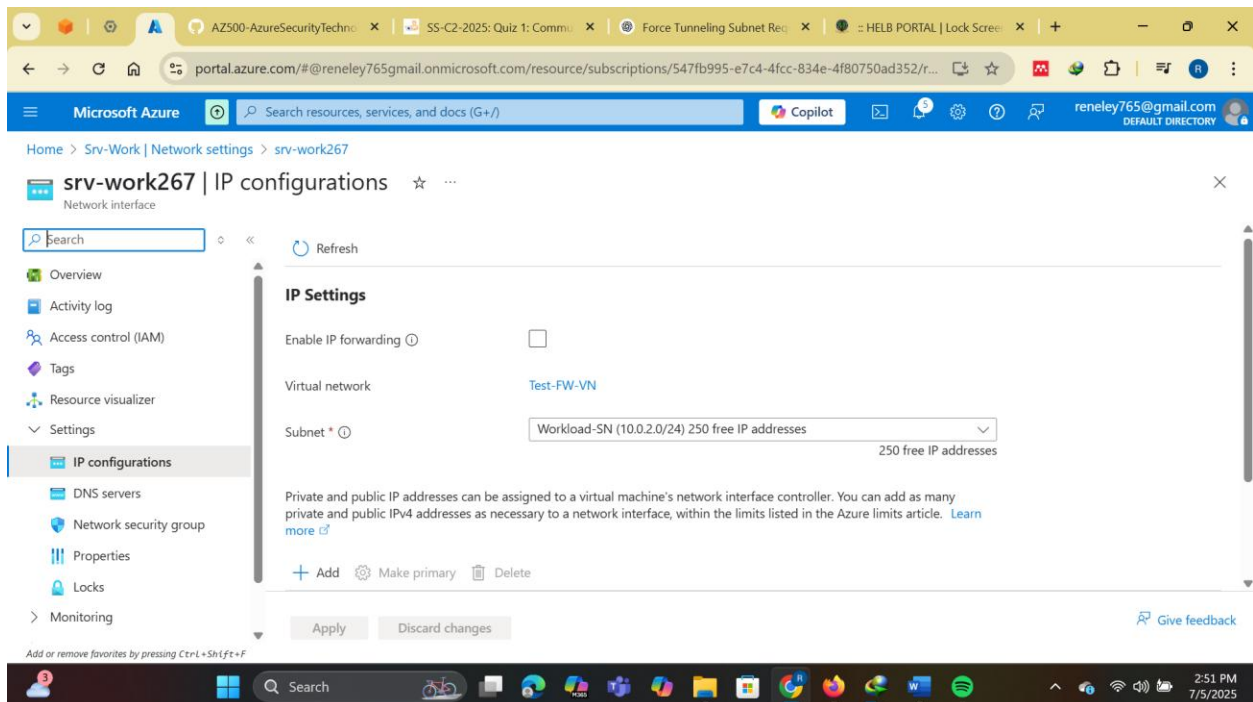


Fig 3.6 Network interface vm is attached to

Inside the NIC blade, I scrolled down to the **DNS servers** option under the **Settings** section. By default, the NIC is set to inherit DNS settings automatically from the virtual network. To ensure that DNS queries are directed through the firewall and use only the specific public DNS servers allowed by the network rule, I switched the DNS configuration from **Default (Azure-provided)** to **Custom**. I then entered **209.244.0.3** as the primary DNS server and **209.244.0.4** as the secondary DNS server, matching exactly the destination IP addresses specified in the firewall's network rule for DNS. Once I confirmed the entries were correct, I clicked **Save** to apply the new DNS configuration to the NIC.

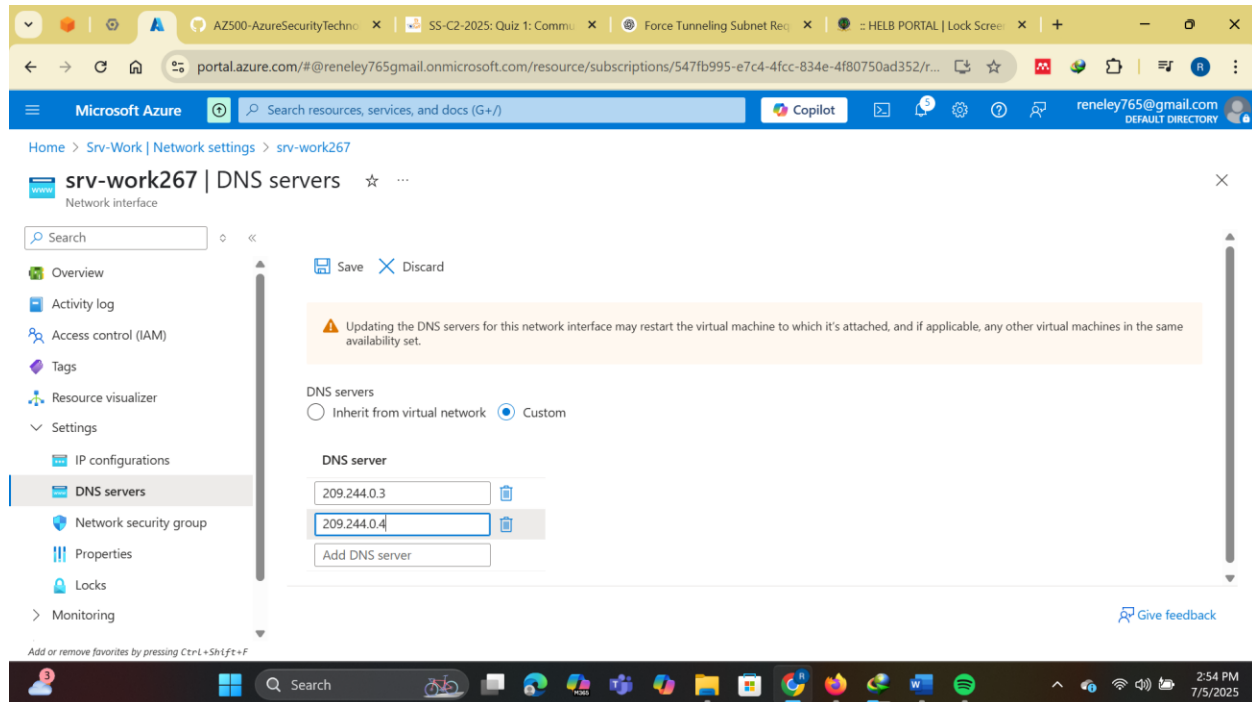


Fig 3.7 Adding dns servers

Because changing the DNS server settings on a NIC in Azure typically triggers a restart of the associated virtual machine, I watched as the **Srv-Work** VM status transitioned to **Updating** and then **Running** again, ensuring the settings were properly applied without any manual restart needed. With this, I made sure that the workload VM would resolve domain names using only the approved external DNS servers, and all queries would flow through the firewall as designed, enforcing the controlled and secure outbound traffic model I had planned.

Task 7: Testing the firewall

Remaining within the same **AZ500LAB08** resource group in the Azure portal, I located the **Srv-Jump** virtual machine, which resides in the jump host subnet. The purpose of this jump host is to

provide a secure entry point to connect into the internal workload VM and test outbound internet access through the firewall. I selected **Srv-Jump** from the list of resources and opened its overview blade. To connect, I clicked **Connect** at the top of the blade and then chose **RDP** from the drop-down menu. The portal generated and downloaded an RDP file for me to use.

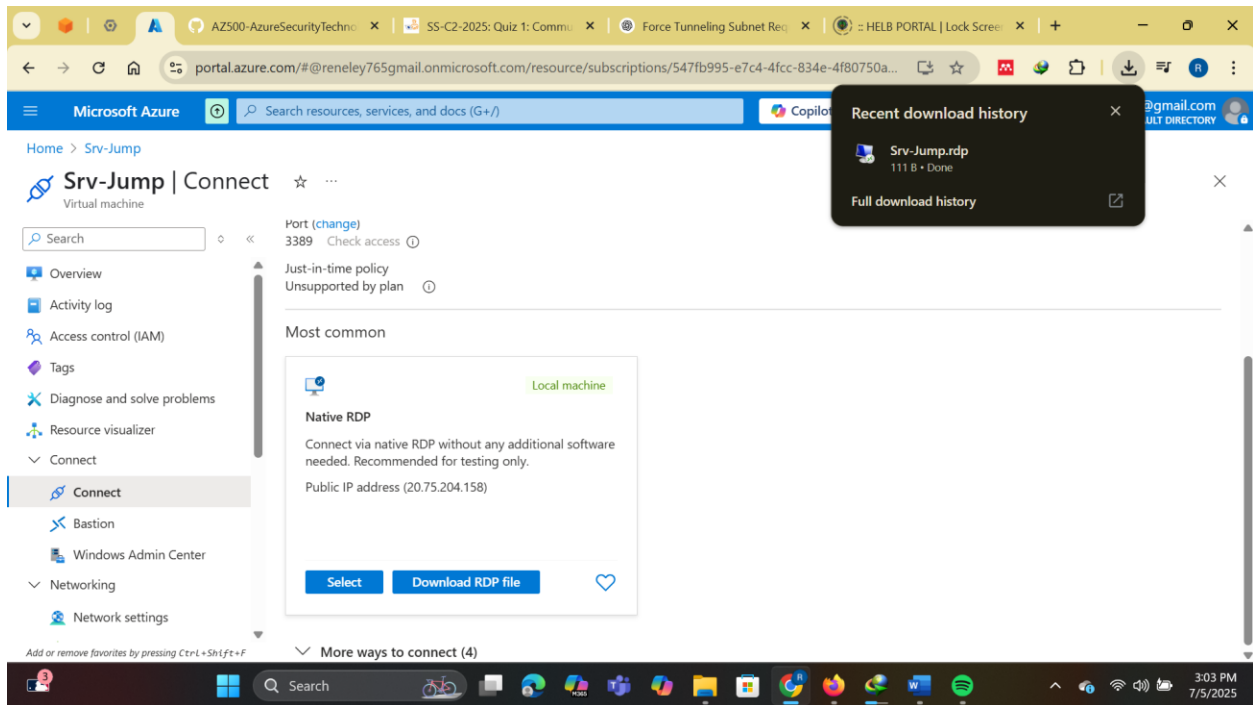


Fig 3.8 Downloading rdp file

I opened the RDP file on my local machine and, when prompted, I authenticated using the local admin account **localadmin** and the secure password I had set during the ARM template deployment in Task 1. Once inside the **Srv-Jump** virtual machine's desktop session, I prepared to access the workload virtual machine from within this internal environment.

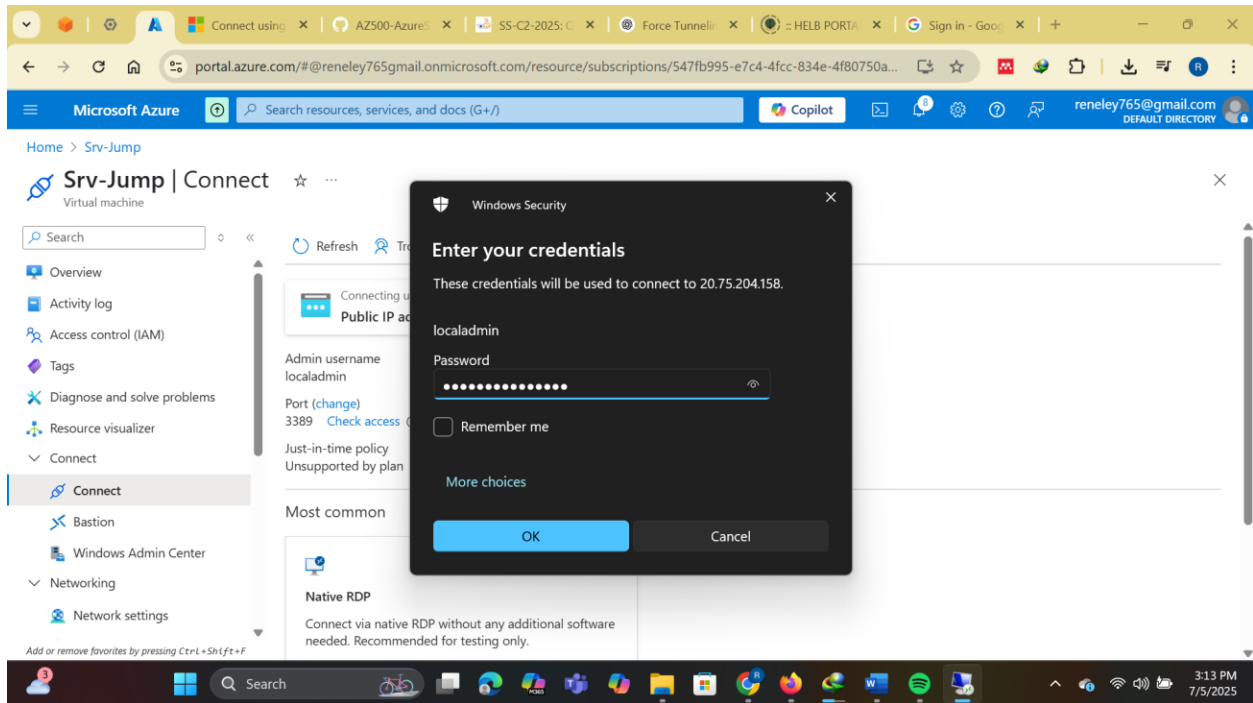


Fig 3.9 Accessing the (jump) machine via rdp

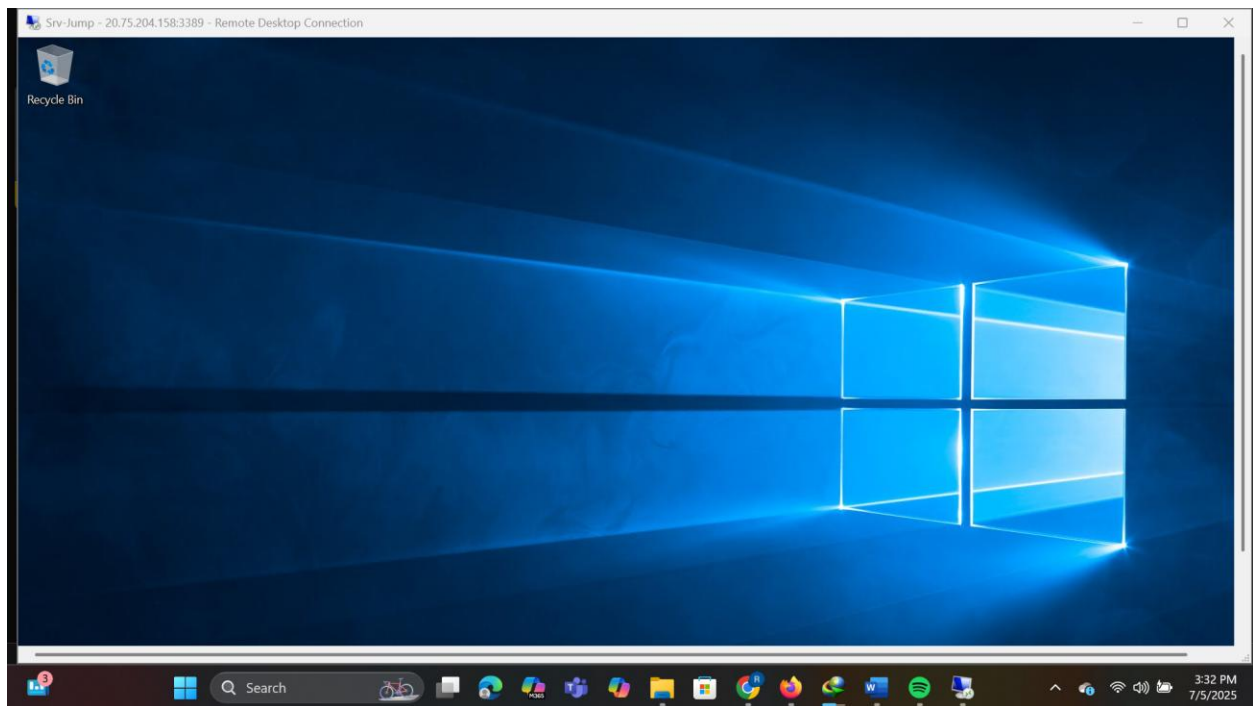


Fig 4.0 Jump srv

Inside the remote session, I opened the **Run** dialog by right-clicking **Start** and choosing **Run**, then typed `mstsc /v:Srv-Work` to open another Remote Desktop Connection directly to the **Srv-**

Work VM. I authenticated again with the same **localadmin** account and password. After logging into **Srv-Work**, I waited a few moments for the Server Manager dashboard to appear.

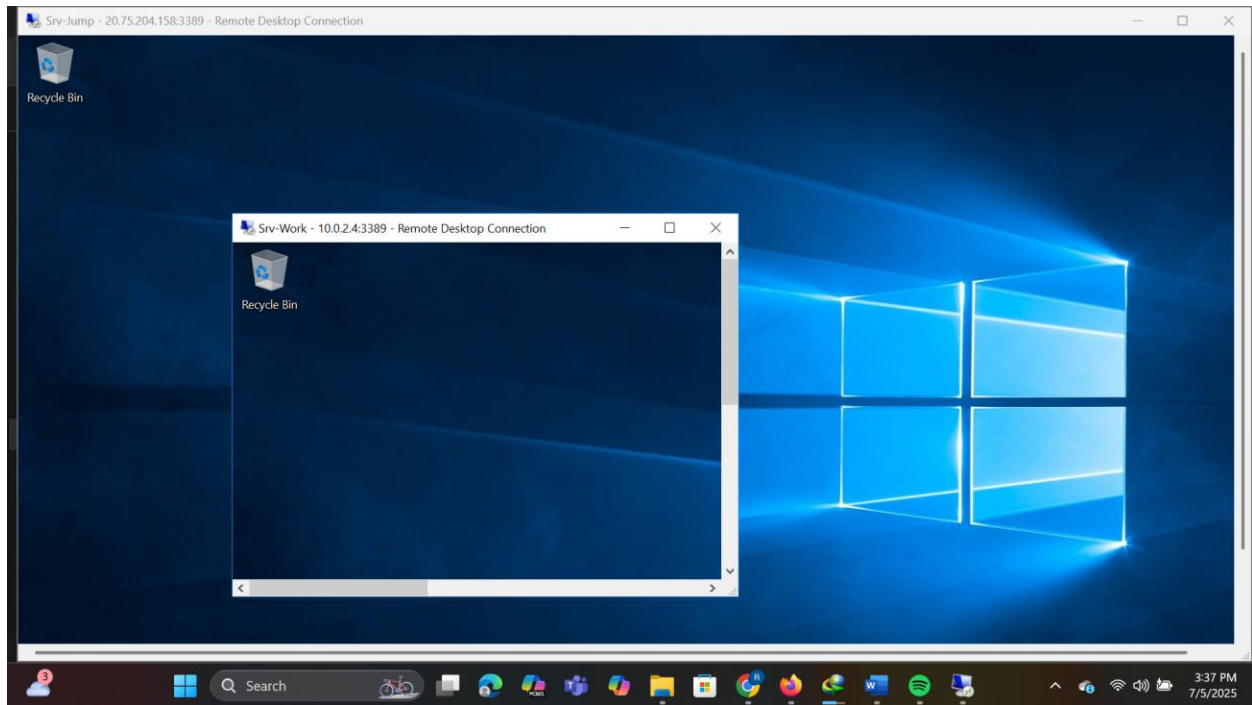


Fig 4.1 Accessing srv work while on srv jump

Before testing internet connectivity, I made sure that Internet Explorer Enhanced Security Configuration was disabled. I navigated within **Server Manager** to the **Local Server** section, found the **IE Enhanced Security Configuration** setting, and clicked the link to open its configuration window. In the dialog box that appeared, I turned off IE Enhanced Security Configuration for both administrators and users, then saved the settings. This was necessary to ensure that my test browsing sessions would not be blocked or redirected unnecessarily by restrictive security settings within the server's default browser.

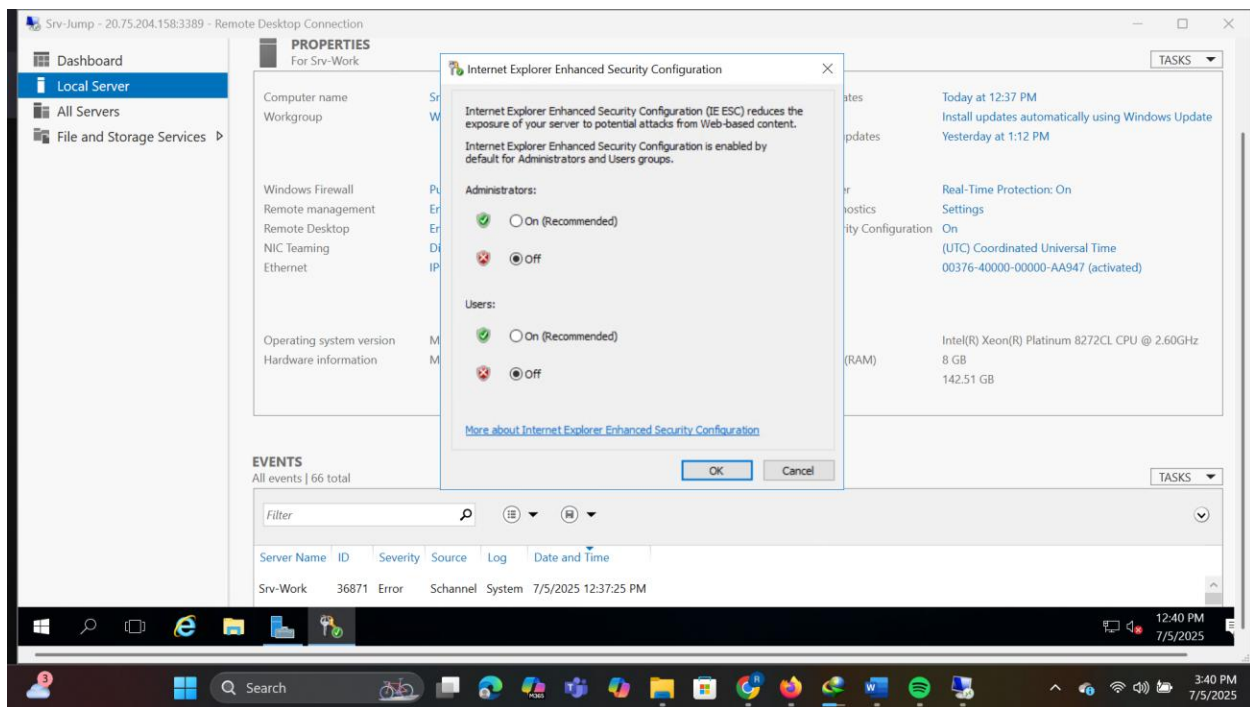


Fig 4.2 Disabling internet explorer enhanced security

With the browser ready, I launched Internet Explorer and entered <https://www.bing.com> in the address bar. The page loaded successfully, confirming that the firewall's application rule permitting outbound HTTP and HTTPS access to www.bing.com was functioning perfectly.

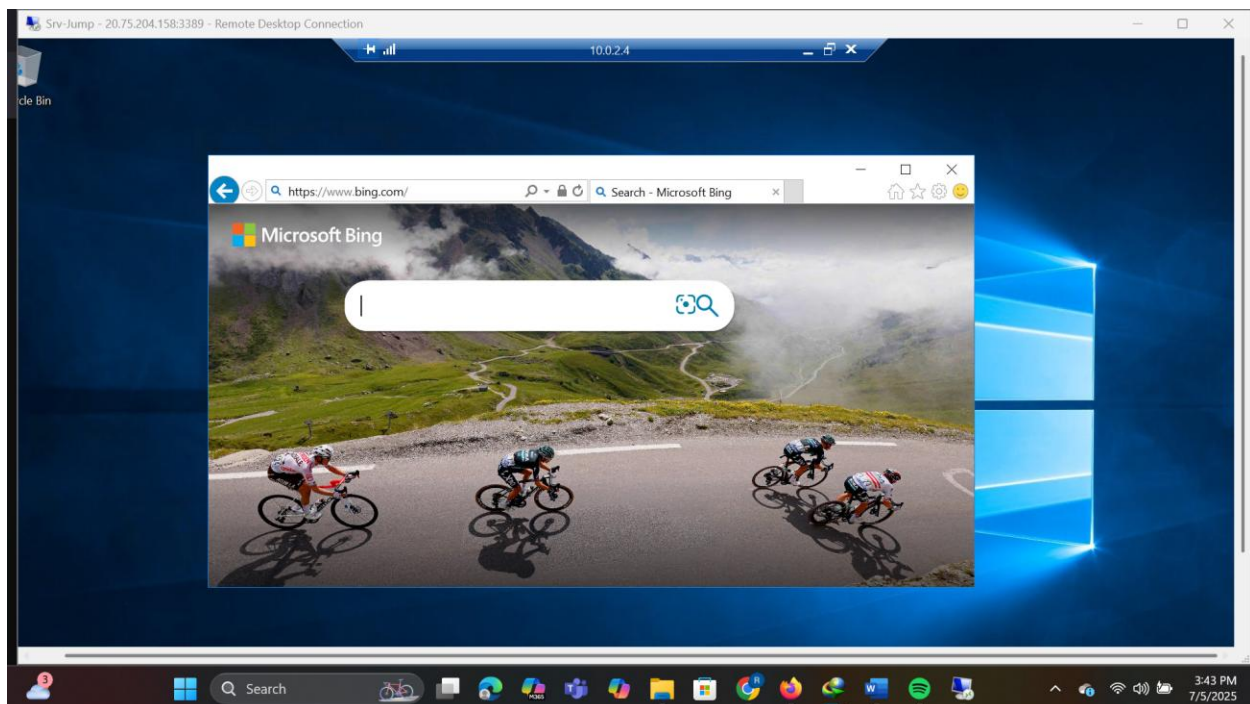


Fig 4.3 Firewall application rule permitting access to bing.com

To test whether the firewall correctly blocked other traffic, I then entered <http://www.microsoft.com> in the address bar. As expected, the connection failed and an explicit deny message appeared in the browser, indicating that the firewall dropped the request because no rule matched and the default action — deny — was enforced. This confirmed that outbound internet traffic was strictly controlled and only allowed destinations explicitly permitted by the firewall's rules could be reached.

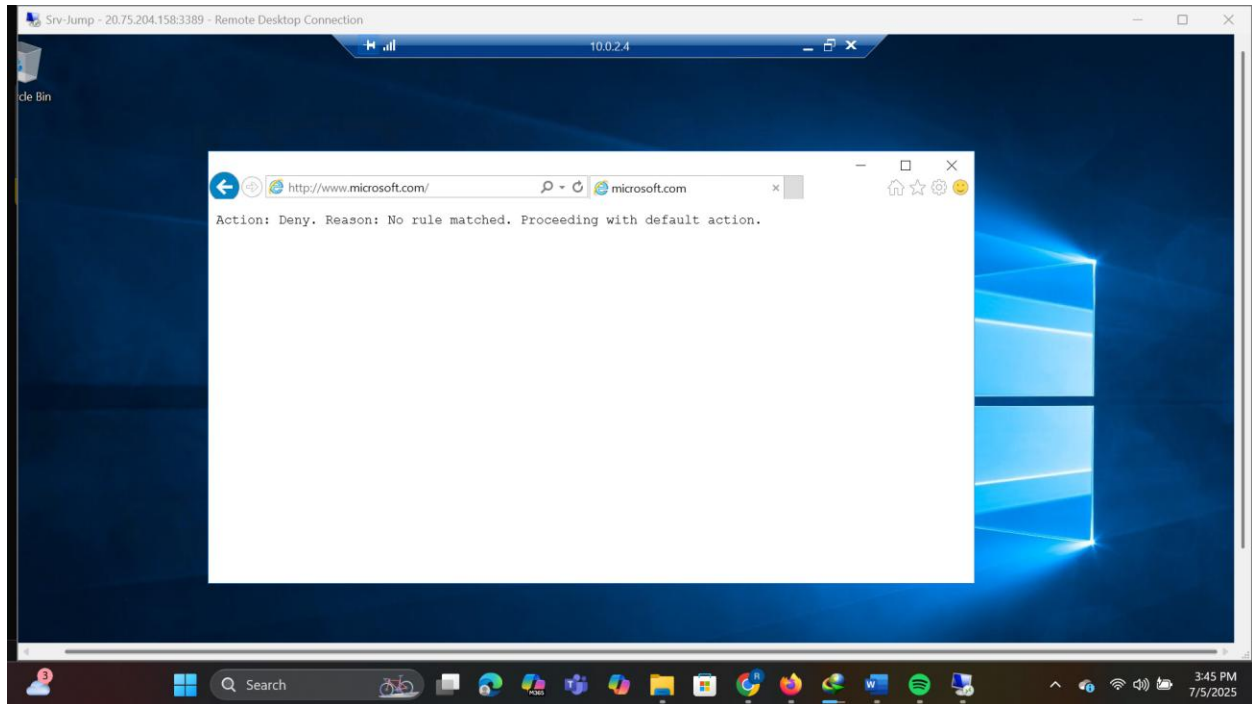


Fig 4.4 Failed connection to Microsoft

With these verifications complete, I closed Internet Explorer, logged off the **Srv-Work** session, and then disconnected from the **Srv-Jump** session on my local machine.

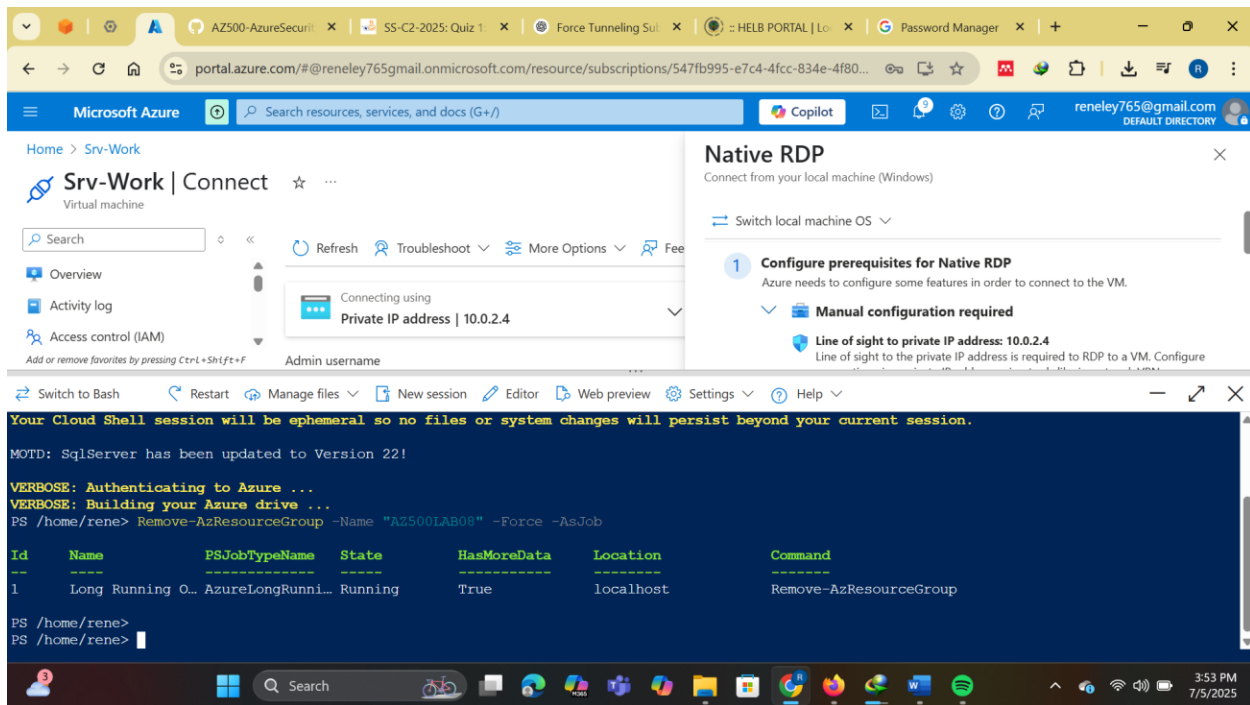


Fig 4.5 Cleaning resources

This concluded my hands-on confirmation that the Azure Firewall, custom route, application rule, and network rule all worked together as designed to secure and filter network traffic between the workload subnet and the internet.

Conclusion

This lab demonstrated how Azure Firewall provides a robust, centralized way to control and secure both outbound and inbound network traffic for resources within an Azure virtual network. By deploying the firewall, setting up a custom route to ensure all outbound traffic from the workload subnet flows through it, and configuring precise application and network rules, I was able to enforce strict, granular controls that allowed access only to approved destinations like www.bing.com and trusted DNS servers. This approach showcased how Azure Firewall supports a Zero Trust model by denying all unspecified traffic and only permitting explicitly allowed communication paths.

Through testing, I confirmed that these security measures function as intended and that the firewall effectively blocks unauthorized requests while enabling legitimate ones. Overall, this lab reinforced how Azure Firewall can serve as a scalable, cloud-native security solution for protecting modern cloud workloads, improving visibility, and maintaining compliance across distributed environments.